

EABench: A Configurable End-to-End Platform for Evaluating Enterprise LLM Agents

HAOYI XIONG, Microsoft Corporation, China

YI LIU, Microsoft Corporation, China

NAN CHEN, Microsoft Corporation, China

BIN ZHU, Microsoft Corporation, China

ZHONGXIAN JIA, Microsoft Corporation, China

YIQIN YU, Microsoft Corporation, China

YI WANG, Microsoft Corporation, China

Enterprise deployment of LLM agents demands more than a static benchmark: researchers need a controllable environment that can generate realistic organizational data, instantiate alternative agent architectures, synthesize challenging evaluation cases, and expose failures in a form that is easy to debug. Existing agent benchmarks capture portions of this problem, but they typically fix the underlying dataset, hard-code the evaluation metrics, or omit the access-controlled and cross-modal structure that defines enterprise knowledge work. We present **EABench**, an end-to-end experimental platform organized around four contributions. First, EABench provides a unified pipeline that connects tenant generation, agent execution, and evaluation inside a single reproducible sandbox. Second, it exposes extreme configurability: models, embeddings, prompts, tools, flow strategies, and sandbox backends are all hot-swappable without code changes. Third, it introduces configurable evaluation-dataset generation: the framework automatically produces needle-in-a-haystack retrieval cases, multi-step reasoning cases, and deep-dive report cases, each paired with user context, ground-truth entities, and weighted natural-language assertions. The evaluation harness is likewise configurable, allowing researchers to redefine judge prompts and scoring thresholds without code changes. Fourth, it supports scenario-aware enterprise simulation with multi-tenant narratives, user-level access control, and an interactive debugging workflow that surfaces tool traces, retrieval evidence, and citations. Together, these design choices make EABench a practical platform for systematic, reproducible study of enterprise agents under realistic operational constraints.

CCS Concepts: • **Computing methodologies** → **Artificial intelligence**; **Machine learning**.

Additional Key Words and Phrases: LLM agents, enterprise AI, benchmarking, evaluation frameworks, synthetic data generation

Authors' Contact Information: Haoyi Xiong, Microsoft Corporation, Beijing, China; Yi Liu, Microsoft Corporation, Beijing, China; Nan Chen, Microsoft Corporation, Beijing, China; Bin Zhu, Microsoft Corporation, Beijing, China; Zhongxian Jia, Microsoft Corporation, Beijing, China; Yiqin Yu, Microsoft Corporation, Beijing, China; Yi Wang, Microsoft Corporation, Beijing, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

ACM Reference Format:

Haoyi Xiong, Yi Liu, Nan Chen, Bin Zhu, Zhongxian Jia, Yiqin Yu, and Yi Wang. 2026. EABench: A Configurable End-to-End Platform for Evaluating Enterprise LLM Agents. *ACM Trans. Intell. Syst. Technol.* 1, 1, Article 1 (April 2026), 36 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Enterprise agents operate in an environment that is substantially harder than the public-web and API-centric settings emphasized by most existing benchmarks. A useful enterprise assistant must resolve people and permissions, search across heterogeneous artifacts, reconstruct event timelines from partial evidence, and synthesize answers that remain grounded in access-controlled sources. These demands are compounded by multi-hop reasoning requirements [19]: enterprise queries rarely have single-document answers but instead require chaining evidence across emails, meeting transcripts, files, and calendar events—precisely the kind of cross-document knowledge retrieval that retrieval-augmented generation (RAG) [8] has made feasible for open-domain question answering, but which enterprise deployments demand at organizational scale under strict permission boundaries. Together, these properties make enterprise evaluation fundamentally a systems problem: the benchmark must model data generation, retrieval infrastructure, execution policy, and assessment criteria together rather than as isolated components.

The urgency of this challenge mirrors the rapid pace of enterprise AI adoption. Commercial deployments such as Microsoft 365 Copilot [11] already process millions of knowledge-work queries daily, yet independent benchmarking is severely hampered by the absence of controlled environments that can reproduce access-controlled organizational data at scale. Even the most capable frontier models struggle in this regime: the OfficeQA Pro benchmark [13]—which requires grounded reasoning over a large, heterogeneous document corpus—found that frontier agents achieve under 12% accuracy when relying on web access alone and under 35% even when given direct corpus access, confirming that enterprise-grade evaluation remains far from solved. General-purpose benchmarks such as GAIA [10], OSWorld [18], and SWE-bench [7] have pushed the frontier of multi-step AI assistance in web navigation, desktop GUIs, and software repositories, but they abstract away the access-controlled, cross-modal, and temporally structured character of real enterprise knowledge work.

Alongside these general-purpose efforts, enterprise-specific benchmarks—WebArena [24], AgentBench [9], ToolBench [14], WorkArena [5], EnterpriseBench [16], DRBench [1], and HERB [3]—each capture important slices of agent capability, but they leave three recurring gaps for enterprise research. First, they generally rely on static datasets or fixed task pools, which prevents researchers from generating new organizational histories or stress-testing a method under controlled scenario variation; while dynamically generated benchmarks have been proposed [15], they address only narrow task domains and do not generalize to multi-source enterprise knowledge work. Second, they rarely expose the agent runtime and the judge as independently configurable objects, making systematic ablations across models, prompts, tools, embeddings, and scoring criteria unnecessarily expensive. Third, they under-specify the debugging problem: when an enterprise agent fails, researchers need to inspect the search trajectory, tool usage, citations, and permission context rather than only the final answer.

EABench addresses these gaps through four tightly coupled contributions. The first contribution is a unified end-to-end platform that connects synthetic tenant generation, agent execution, and evaluation in a

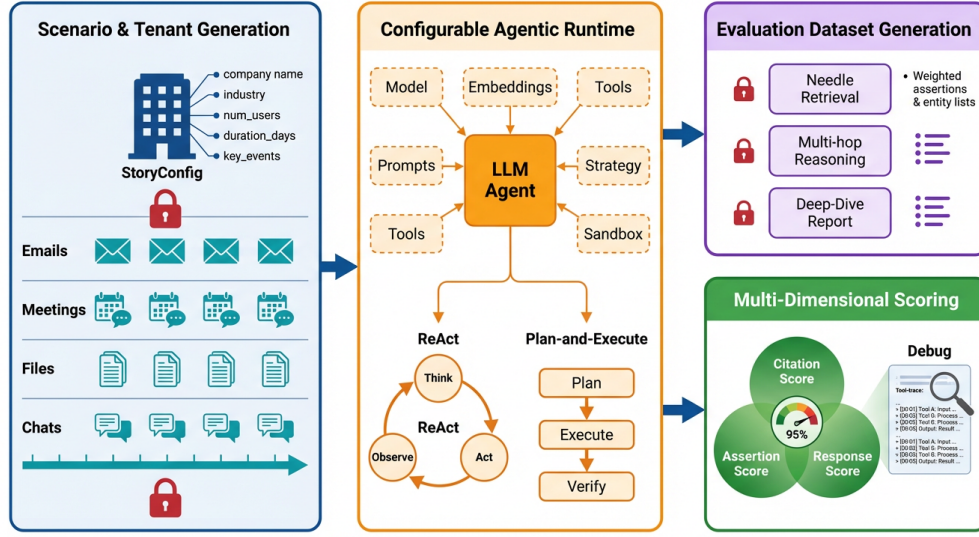


Fig. 1. EABench graphical overview. The four coupled platform stages proceed from synthetic tenant generation (Stage 1), through configurable agent execution (Stage 2) and automated evaluation-dataset construction (Stage 3), to multi-dimensional scoring with interactive debugging (Stage 4), forming a single reproducible experimental pipeline.

single reproducible pipeline. The second is extreme configurability: six major runtime dimensions—models, embeddings, cognitive architectures, prompts, tools, and sandbox implementations—are all hot-swappable without source-code edits. The third is configurable query generation and evaluation, in which the framework automatically generates three query types and pairs them with user-specific access context, ground-truth entity lists, weighted assertions, and judge prompts that can themselves be redefined without code changes. The fourth is scenario-aware enterprise simulation, which varies company size, industry, and event structure while supporting interactive debugging over the same runtime used for batch evaluation.

This design changes the role of a benchmark from a fixed leaderboard to an experimental sandbox. A researcher can generate a legal-services tenant and a software-incident tenant from the same pipeline, evaluate ReAct and plan-and-execute agents under identical access constraints, swap only the embedding model or only the citation judge, and then inspect the full tool trace when performance changes. Table 1 summarizes this shift relative to prior work.

The rest of the paper develops these ideas in order. Section 2 positions EABench relative to prior agent benchmarks and enterprise retrieval systems. Section 3 formalizes the end-to-end platform architecture, the configurable runtime, the data-generation pipeline, the evaluation-dataset generator, the multi-dimensional evaluation harness, and the interactive debugging interface. Section 4 describes the scenario families used to instantiate the benchmark. Section 5 specifies the experimental protocol supported by the framework, including cross-model evaluation, human validation, and ablation studies. Section 6 concludes with limitations and future directions.

Table 1. Comparison of EABench with representative agent benchmarks. ✓ indicates direct support, Partial indicates limited or indirect support, and – indicates that the capability is absent from the published benchmark design.

Feature		WebArena	AgentBench	ToolBench	WorkArena	DRBench	HERB	EnterpriseBench	EABench
End-to-end evaluation pipeline		–	–	–	–	Partial	Partial	Partial	✓
Configurable data generation		–	–	–	–	Partial	✓	Partial	✓
Configurable agent runtime		–	–	–	–	–	–	–	✓
Configurable evaluation metrics		–	–	–	–	Partial	–	–	✓
Auto query + ground-truth generation		–	–	–	–	–	Partial	Partial	✓
Assertion-based fact checking		–	–	–	–	Partial	Partial	–	✓
User-level access control		–	–	–	Partial	Partial	Partial	✓	✓
Interactive debugging		–	–	–	–	–	–	–	✓
Pluggable LLM providers		–	Partial	–	–	✓	✓	Partial	✓
Sandboxed tool execution		–	Partial	–	–	–	–	–	✓

2 Related Work

We review the literature across two primary axes: the evolution of agent benchmarking from general web environments to enterprise-specific tasks, and the enabling technologies behind EABench, including synthetic data generation, cognitive architectures, and automated evaluation metrics.

2.1 Agent Benchmarks

General-purpose benchmarks have been instrumental in driving rapid progress in LLM tool use. However, as discussed below, their reliance on static web environments and generic API puzzles limits their applicability to the messy, permission-gated reality of private organizational networks.

WebArena [24] provides a realistic web browsing environment with four web applications (shopping, reddit, GitLab, map) and 812 long-horizon tasks. While WebArena evaluates task completion in a realistic setting, its tasks are scoped to web interfaces and do not address enterprise knowledge retrieval. **WorkArena** [5] extends this approach to ServiceNow workflows, targeting enterprise service management tasks. However, its data is limited to a single application and does not cover the richly inter-connected multi-source data of a real enterprise.

AgentBench [9] is a multi-dimensional benchmark spanning eight environments including OS, database, web browsing, and lateral reasoning tasks. It demonstrates that current LLMs struggle with multi-step tool-use tasks; however, its environments are largely synthetic or game-like rather than representative of knowledge-worker contexts.

ToolBench [14] focuses specifically on evaluating LLMs’ ability to invoke real-world APIs from a large catalog of 16,000 APIs. While this directly tests tool-use, the tasks are API-centric rather than information-synthesis-centric, and there is no notion of user-level access control or data provenance.

τ -bench [20] introduces the concept of user simulation for interactive task evaluation in airline and retail domains. The approach of simulating realistic user interactions—rather than specifying fully deterministic tasks—is conceptually related to EABench’s user-level evaluation mode, though τ -bench targets customer service scenarios rather than enterprise knowledge work.

Information Worker Benchmarks. While systems such as **WorkArena** [5] or OSWorld provide highly realistic tasks mapped to complex user interfaces, they remain fundamentally inflexible when modifications to the underlying environment or integrations of novel agent architectures are required. Conducting ablation

studies within these robust ecosystems demands significant engineering overhead, restricting iterative cognitive architecture design.

Static vs. Dynamic Environments. **GAIA-2** [4] and **SWE-Bench** [7] push the boundaries of general AI assistance by testing complex, real-world reasoning, multimodal tool use, and sophisticated software engineering tasks. However, these represent completely static and unalterable evaluation paradigms. Their rigid evaluation harnesses prevent isolating targeted variables—such as altering the underlying embedding model used in retrieval tools or limiting exposure to specific operational commands. Conversely, EABench explicitly addresses this via a deeply configurable design, granting researchers an open architectural playground to systematically investigate execution strategies across diverse scenario and runtime configurations.

2.2 Enterprise Agentic Benchmarks

Recognizing the limitations of general domains, recent works have begun simulating enterprise environments. While these benchmarks test multi-hop reasoning over corporate artifacts, they often rely on fixed, non-extensible datasets, leaving a critical gap for highly configurable, generative sandboxes.

DRBench [1] evaluates agents on open-ended, multi-hop deep research tasks that require synthesizing insight-driven reports from a mix of public web data and private company knowledge bases. Grounded in realistic user personas, DRBench spans a heterogeneous search space (including internal chats, cloud file systems, spreadsheets, and emails) and specifically scores an agent’s ability to recall critical insights with proper source citations. Similarly, **HERB** [3] presents a benchmark containing 39,190 enterprise artifacts designed for evaluating deep search and retrieval-augmented generation over heterogeneous, interconnected data sources like documents, meeting transcripts, and Slack messages via synthetic workflows simulating product planning and support stages. While these benchmarks test multi-hop reasoning over enterprise artifacts, EABench distinguishes itself by generating a fully-coherent synthetic timeline, providing a fully configurable agent runtime (ReAct vs. Plan-and-Execute), and strictly verifying factuality via structural citation scoring combined with user-level access control. Teams chats. Its architecture involves retrieval-augmented generation (RAG) over enterprise data with user-level access control. While such systems demonstrate the commercial viability of enterprise agents, they do not provide open evaluation datasets or benchmarking infrastructure.

Glean and similar enterprise search platforms combine vector search with access control lists (ACLs) to implement personalized search across connected SaaS applications. These systems illustrate the importance of user-context-aware retrieval—a feature EABench explicitly supports through its user-level indexing and filtering.

2.3 Synthetic Data Generation for Benchmarks

Real-world corporate data is scarce and constrained by extreme privacy bottlenecks. To overcome this, researchers have turned to LLM-driven synthetic generation. EABench extends these techniques beyond isolated QA pairs to construct fully interconnected, temporally coherent organizational ecosystems.

AgentInstruct [12] uses LLMs to generate instruction tuning data by transforming raw documents through a pipeline of multiple specialized agents. **FrontierMath** [6] demonstrates that LLM-generated problems can challenge state-of-the-art models. **WildChat** [22] collects naturally occurring LLM conversations in the wild.

EABench’s approach differs from prior work in that it generates *structured organizational corpora* rather than isolated question-answer pairs. The generator produces inter-connected narrative artifacts (emails, meetings, files, chats) grounded in a shared storyline, enabling queries that require multi-hop reasoning across document types and temporal reasoning over event sequences.

2.4 LLM-as-a-Judge Evaluation

Evaluating open-ended, multi-step agent trajectories requires scalable metrics that transcend exact-string matching. LLM-as-a-Judge paradigms have emerged as a robust solution, which we adapt to perform granular citation verification and assertion checking while mitigating inherent evaluator biases.

Automated evaluation using LLMs as judges has gained significant traction following MT-Bench [23] and Chatbot Arena [2]. LLM judges show high correlation with human preferences at lower cost and enable evaluation of open-ended responses that resist automatic metrics.

EABench extends the LLM-as-a-Judge paradigm to *agentic evaluation*, where the judge must assess not only the final response but also the *reasoning process*—including whether cited sources are accurate, whether tool calls were appropriate, and whether the overall research plan was followed.

2.5 Cognitive Architectures and Agentic Workflows

The cognitive architecture of an agent fundamentally dictates its capability and operational ceiling. Recent advancements have synthesized rigid chain-of-thought prompting into broader structured agentic workflows, which emphasize iterative refinement, dynamic tool use, and reflection over single-shot generation. Within this landscape, we contrast single-pass reasoning loops with deliberative planning and broader workflow frameworks, all of which EABench treats as first-class, easily configurable strategies for applied scientists.

The ReAct paradigm introduced the concept of interleaving reasoning traces with tool invocations [21]. This approach demonstrated that explicitly reasoning about the next action before taking it substantially improves task performance on knowledge-intensive tasks compared to standard zero-shot execution. Plan-and-Solve agents build upon and extend this fundamental idea by explicitly separating the planning phase, which handles the generation of a high-level task decomposition, from the execution phase [17]. This deliberate separation can significantly improve reliability on complex multi-step tasks by preventing the underlying agent from losing the contextual thread during long and intricate tool-use sequences.

Furthermore, modern agentic workflows have evolved these frameworks by incorporating reflection, multi-turn tool calling, and structured orchestration. These workflows shift the paradigm from monolithic text generation to a composite system where an agent iteratively critiques its intrinsic execution trajectory, adapts to newly retrieved information, and dynamically alters its multi-step plan. EABench natively implements these advanced paradigms as first-class, configurable execution strategies. This deep integration systematically enables the direct comparison of performance characteristics across distinct cognitive architectures and iterative workflows directly within evaluation tasks.

3 Methodology

We now describe the six components of the EABench platform in turn, covering the end-to-end architecture, the configurable runtime, the data-generation pipeline, the evaluation-dataset generator, the multi-dimensional evaluation harness, and the interactive debugging interface.

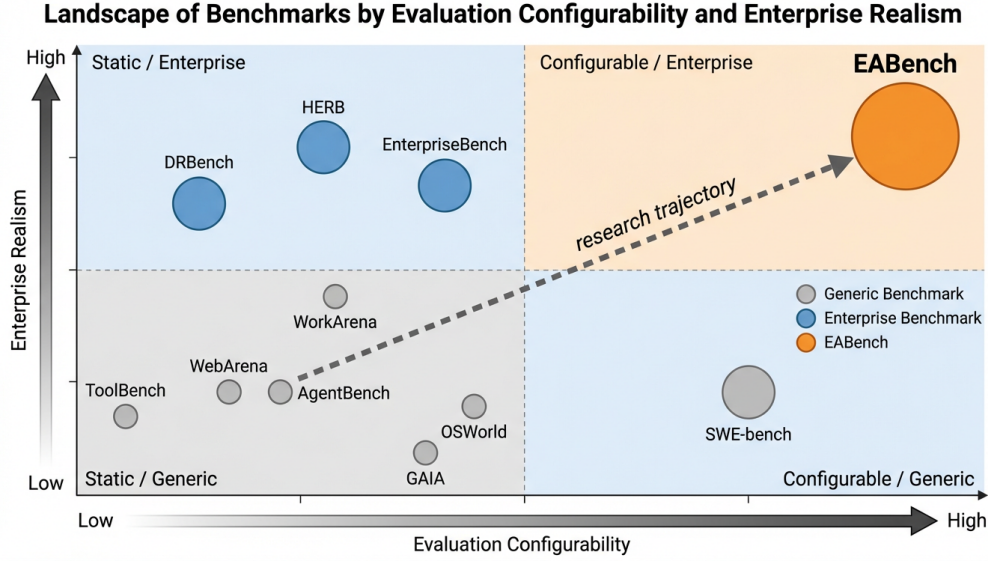


Fig. 2. Positioning of representative agent benchmarks along two axes: enterprise realism (y-axis) and evaluation configurability (x-axis). Bubble size reflects the number of first-class benchmark features supported (cf. Table 1). EABench uniquely occupies the top-right quadrant, combining scenario-aware organizational simulation with end-to-end configurability across all platform dimensions.

3.1 End-to-End Platform Architecture

EABench is organized as a single experimental pipeline rather than a loose collection of scripts. A scenario specification begins with a **StoryConfig**, which defines the company profile, timeline, employee count, and key events that drive narrative evolution. The tenant generator expands this specification into a multi-modal enterprise corpus containing users, emails, chats, meetings, and files. The same tenant is then consumed by the agent runtime, which executes under a concrete user identity and records its full tool trajectory. Finally, the evaluation harness scores the resulting answer with both deterministic and judge-based metrics. Because each stage writes explicit artifacts to disk, the entire process is reproducible and inspectable.

This architecture supports two complementary use modes. In batch mode, researchers generate a tenant, evaluate one or more agent configurations, and compare the resulting metric distributions. In interactive mode, they run the same runtime against the same tenant but inspect intermediate search results, tool calls, citations, and user context while debugging failure cases. The shared backend is important: debugging traces come from the exact runtime that produces benchmark results, so qualitative inspection and quantitative evaluation remain aligned.

The key systems claim of EABench is therefore not only that each subsystem is configurable, but that the interfaces between subsystems are stable and explicit. A change in the generator alters the tenant and the resulting query set; a change in the runtime alters the trajectory and answer; a change in the judge alters the scoring layer. By exposing these seams directly, EABench makes causal experimentation possible.

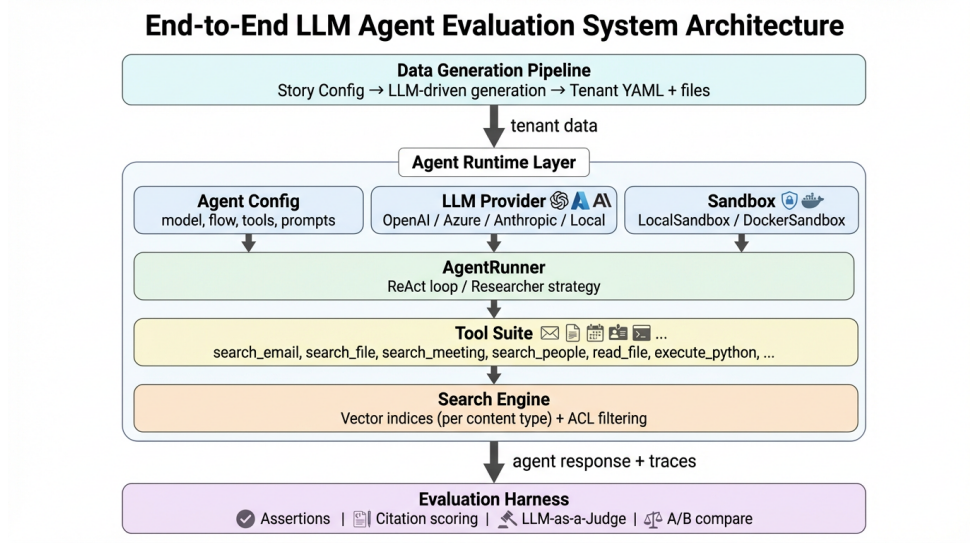


Fig. 3. End-to-end EABench architecture. The platform connects scenario specification, tenant generation, configurable runtime execution, evaluation-case generation, and interactive debugging in one reproducible workflow.

Table 2. Primary artifacts exchanged across the EABench pipeline.

Stage	Primary artifact	Role in the pipeline
Scenario specification	StoryConfig and prompt templates	Defines the enterprise domain, timeline, narrative drivers, and generator behavior
Tenant generation	tenant.yaml, generation_log.json, modality-specific YAML files, sandbox files	Materializes a coherent enterprise workspace with explicit provenance
Evaluation dataset generation	eval_dataset_*.yaml	Converts raw tenant history into query cases with user_id, entity_list, and weighted assertions
Agent runtime	Response text, tool traces, token and latency metrics	Executes a concrete agent configuration under a specific access context
Evaluation harness	Per-case JSON results and aggregate summaries	Produces assertion, citation, and operational diagnostics for comparison and analysis
Interactive debugging	Search evidence, tool-call logs, citation traces	Explains success and failure modes on individual cases without changing the runtime

3.2 Configurable Agentic Runtime

3.2.1 YAML Configuration Schema. The runtime follows a configuration-as-code model in which each agent variant is represented by a single YAML file. This file specifies the reasoning model, embedding model, system prompt, optional planning prompt, per-tool query-analyzer prompts, tool exposure, and flow strategy. The design choice is deliberate: researchers should be able to version-control an agent definition, rerun it on a new tenant, and attribute behavioral changes to the configuration diff rather than to hidden code edits.


```

1 id: react-agent-v1
2 version: "1.0.0"
3 model:
4   provider: azure
5   name: gpt-4o
6   parameters: {temperature: 0.7}
7 embedding:
8   provider: azure
9   model: text-embedding-ada-002
10 system_prompt: |
11   You are an intelligent enterprise assistant.
12   {user_profile}
13 query_analyzer_prompt:
14   search_email: |
15     Analyze the user's email query and return
16     strategy, refined_query, and sender_name.
17 tools:
18   definitions:
19     - search_email
20     - search_file
21     - search_meeting
22     - search_people
23     - read_file
24     - execute_python
25 flow:
26   strategy: react
27   max_turns: 5

```

Fig. 4. Condensed runtime configuration illustrating how model choice, prompt behavior, tool exposure, and flow strategy are all declared in YAML.

Table 3. Six runtime dimensions that can be changed without source-code edits.

Dimension	Configuration field	Research question enabled
Reasoning model	<code>model.*</code>	How sensitive is performance to the underlying LLM family and decoding parameters?
Embedding model	<code>embedding.*</code>	How much of end-to-end performance is driven by retrieval quality rather than reasoning quality?
Cognitive architecture	<code>flow.strategy</code>	Does ReAct or plan-and-execute better handle multi-step enterprise tasks?
Global prompting	<code>system_prompt,</code> <code>planning_prompt</code>	How do role instructions and planning structure affect behavior and citation quality?
Tool exposure	<code>tools.definitions</code>	Which tools are necessary for each query type, and what are the failure modes when they are removed?
Sandbox implementation	tool dependencies and runtime backend	How do local, remote, or more restrictive execution environments change tool use and latency?

3.2.2 Cognitive Architectures. EABench currently exposes two cognitive architectures. Under the `react` strategy, the runtime loops through thought, tool call, observation, and response generation until the model emits a final answer or reaches a hard turn limit. Under the `researcher` strategy, the runtime first invokes a planner to produce a structured plan and then executes that plan with the normal tool-enabled loop. The

Table 4. Comparison of the two built-in runtime strategies.

Strategy	First action	Typical strength	Typical failure mode
ReAct	Immediate tool-enabled reasoning	Fast response on local retrieval tasks	Can drift on long dependency chains or overlook needed decomposition
Researcher	Tool-free planning	Stronger structure on multi-step tasks	Higher latency and possible over-planning on simple cases

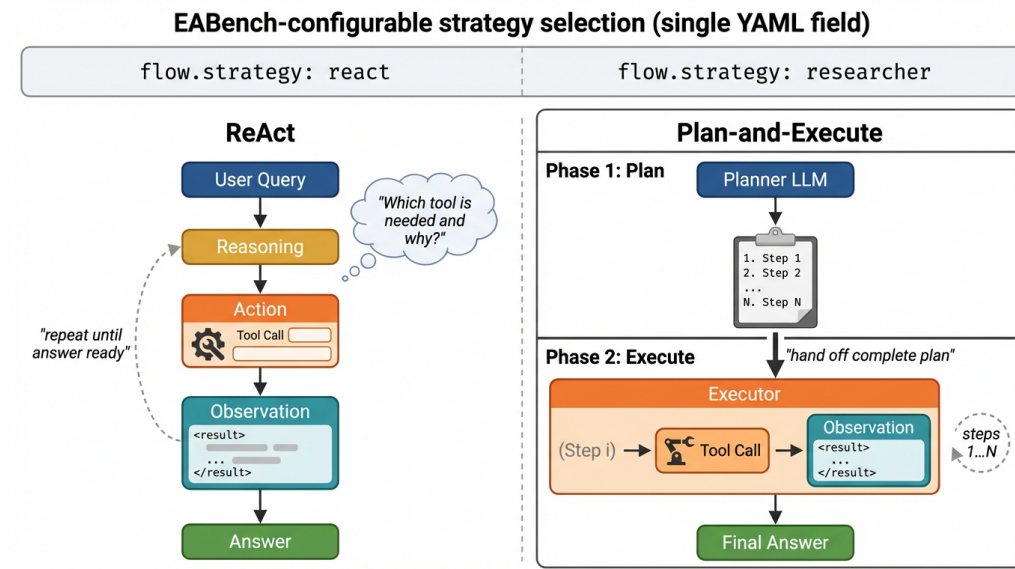


Fig. 5. The two cognitive architectures natively supported by EABench as hot-swappable `flow.strategy` values. ReAct (left) interleaves reasoning and acting in a tight feedback loop; Plan-and-Execute (right) separates a global planning phase from sequential step-wise execution, reducing context drift on long multi-hop tasks.

second strategy adds overhead, but it is often better suited to multi-hop cases in which the agent must identify a dependency chain before any single tool call is obviously useful.

3.2.3 Multi-Provider LLM Support. The runtime separates the configuration schema from the provider implementation through a common LLM interface. Provider choice is driven by the YAML field `model.provider`, while credentials and deployment-specific settings are resolved from environment variables. This separation makes provider swapping operationally cheap: researchers can move between OpenAI, Azure-hosted deployments, or compatible local endpoints while preserving the rest of the runtime definition. That property is especially important for cross-model evaluations, because it prevents provider migration from becoming a hidden code change.

3.2.4 Enterprise Tool Suite and Search Infrastructure. The built-in tool suite combines semantic retrieval tools with sandboxed execution tools. Search tools cover emails, files, meetings, people, one-to-one chats, group chats, and channel messages. Execution tools cover file inspection, directory listing, shell execution, and

Table 5. Core fields in the StoryConfig specification.

Field	Type	Purpose
company_name	string	Names the tenant and appears in prompts and output paths
industry	string	Conditions vocabulary, artifacts, and domain-specific events
company_size	string	Influences organizational breadth and communication patterns
num_users	int	Sets the size of the user roster and social graph
duration_days	int	Controls the temporal window over which history accumulates
key_events	list[string]	Defines narrative milestones that should recur across modalities
description	string	Supplies free-form scenario context to the generator prompts

Python execution. Tools are registered through a decorator-based registry that automatically exports JSON schemas for model-facing tool calling, while dependencies such as the sandbox, search engine, and query analyzer are injected by the runner. This keeps tool implementations modular and makes tool ablations straightforward: removing a tool from the YAML file is sufficient to change the agent’s action space.

Retrieval itself is implemented as a set of separate vector indices rather than as one monolithic store. EABench indexes at least nine content collections, including file snippets, full file contents, emails, chats, group chats, channels, meeting metadata, meeting transcripts, and user profiles. Similarity is computed with cosine similarity, and the resulting embeddings are cached on disk to avoid recomputation across repeated evaluations. The indices are filtered by user context before results are returned, which means the runtime and the evaluator both observe the same access-controlled search space.

3.2.5 Query Analyzer. An optional LLM-driven query analyzer sits between the user request and the retrieval tools. Instead of forwarding raw language directly to the vector index, the analyzer can refine a query, select a search strategy, or resolve a sender filter before retrieval occurs. Because its prompts are themselves configurable in YAML, EABench treats search refinement as a first-class experimental variable. Researchers can therefore study not only whether a search tool is useful, but whether specific decomposition heuristics or query rewrites materially improve retrieval fidelity.

3.3 Scenario-Aware Synthetic Data Generation

3.3.1 StoryConfig and Scenario Specification. Synthetic tenant generation begins with a **StoryConfig** that defines the company profile and the narrative arc the generator must realize. The configuration includes the company name, industry, company size, number of users, duration of the simulation, and a list of key events that anchor the narrative. This representation is compact enough for rapid scenario design yet expressive enough to vary organizational complexity and event density. The same generator can therefore instantiate a university administration scenario, a legal-services scenario, or a software-incident scenario without changing the code path.

3.3.2 Five-Stage Generation Pipeline. The generator executes in five stages. It first creates the directory scaffold and initializes modality-specific YAML files. It then generates the user roster in batches, using diversity prompts to avoid name collisions and departmental imbalance. Next, it iterates day by day through the simulation, producing 3–5 narrative events per day while maintaining rolling history buffers. It materializes

those events as emails, chats, meetings, and files, and it finally writes a complete `generation_log.json` that serves as the source material for evaluation-case generation. The pipeline is incremental by design: artifacts are appended to disk as they are generated, which supports large tenants without requiring all content to remain in memory.

Table 6. Five-stage tenant-generation process.

Stage	Name	Output and purpose
1	Directory scaffold and initialization	Creates tenant directories, empty modality YAML files, and a persistent workspace for generated documents
2	User generation with diversity control	Produces the employee roster, titles, departments, managers, skills, and group memberships while avoiding duplicates
3	Daily narrative generation	Expands key events into day-level story beats using recent-history and long-history buffers to preserve temporal coherence
4	Artifact materialization and cross-referencing	Generates emails, chats, meetings, and files whose content explicitly references shared events and decisions
5	Evaluation-log export	Serializes the full event history into <code>generation_log.json</code> for downstream query generation and auditability

3.3.3 Content Type Generation. Each artifact type is generated with a schema tailored to its role in enterprise communication. Emails are produced in two steps: a summary pass proposes sender, recipients, subject, and context, and a second pass expands that summary into realistic prose. Chats are likewise generated from conversation summaries into timestamped message sequences. Meetings produce three linked artifacts: metadata, transcript, and sidebar chat. Files are created as both metadata entries and concrete documents inside the sandboxed file tree. This staged process gives the generator enough structure to maintain consistency while still allowing varied surface realizations.

3.3.4 Narrative Coherence Mechanisms. Long-range coherence is preserved through explicit memory structures rather than through prompt length alone. A recent-history buffer stores the last several days of events verbatim, while a long-history buffer stores compressed summaries of earlier periods. Every seven simulated days, the recent buffer is summarized and appended to the long-term history. This mechanism allows later generations to mention prior incidents, decisions, and unresolved tensions without forcing the model to ingest the entire raw history each day. Cross-document references are then encouraged directly in the prompts, so that an email can cite a meeting outcome, a file can summarize a discussion from chat, and a report can reference both.

3.4 Configurable Evaluation Dataset Generation

3.4.1 Three Query Types. EABench does not treat evaluation cases as fixed annotations authored by hand. Instead, it derives them automatically from the generated tenant history, producing three query types that cover retrieval, reasoning, and synthesis. Needle-in-a-haystack cases ask the agent to locate a specific artifact or a very small set of artifacts without ever naming internal identifiers explicitly. Multi-step reasoning cases require the agent to follow a dependency chain across multiple modalities, such as resolving a person in one artifact and then using that reference to locate the decisive meeting or file. Deep-dive report cases ask for a

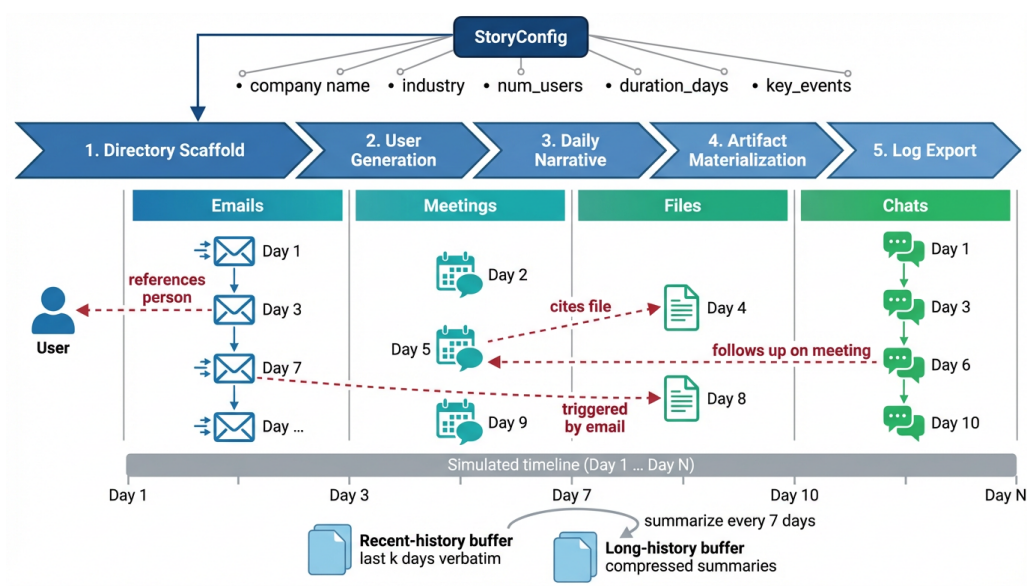


Fig. 6. EABench five-stage tenant-generation pipeline and inter-artifact linkage graph. A single StoryConfig drives the parallel creation of emails, meetings, files, and chats, which are connected by explicit cross-document references along a shared temporal timeline. These cross-references are what enable multi-hop and temporal reasoning evaluation cases.

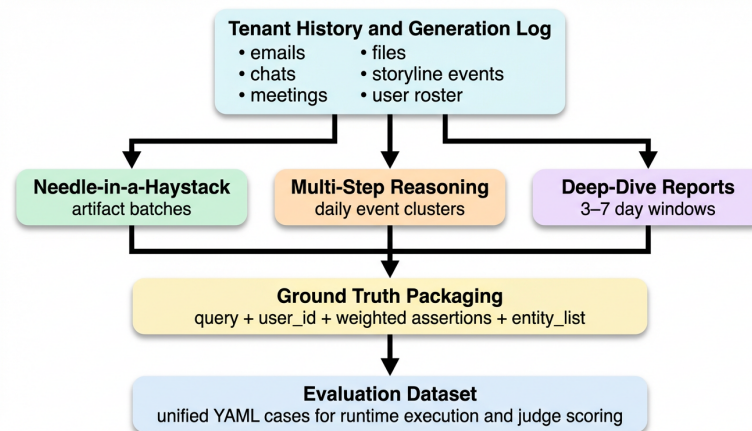


Fig. 7. Evaluation-case generation pipeline. The same tenant history is re-sliced into retrieval, reasoning, and report-generation cases, each with explicit user context and ground truth.

coherent summary of an incident or initiative spread across several days of activity. This tripartite design ensures that evaluation does not collapse to a single notion of success.

3.4.2 Ground Truth Association. Every generated case carries two forms of ground truth. The first is a structured `entity_list`, which records the concrete artifacts that a correct solution should retrieve. The second is a set of natural-language assertions that describe what the answer must contain. Each case is also

Table 7. Automatically generated query types in EABench.

Query type	Typical form	Primary benchmarked capability
Needle-in-a-haystack retrieval	Request to find a specific email, file, meeting, or message using natural constraints such as date, author, topic, or artifact type	Precision retrieval under realistic phrasing and user-level access control
Multi-step reasoning	Request whose answer depends on chaining evidence across multiple artifacts, often across different modalities	Decomposition, dependency tracking, and selective tool use
Deep-dive report generation	Request for a briefing, incident summary, or thematic report over a multi-day window	Long-form synthesis, temporal organization, and citation-grounded summarization

bound to a specific `user_id`, which means the case is evaluated under an explicit access-control context rather than against a globally visible corpus. This design is important because it distinguishes “the answer is wrong” from “the answer is unavailable to this user.”

3.4.3 Assertion Generation for Fact Checking. Assertions are generated automatically from the same context used to formulate the query. They are written in natural language, may be weighted by importance, and focus on factual obligations rather than stylistic preferences. A search case might require the answer to name the correct sender and subject line, while a report case might require the answer to mention the triggering event, the main decision, and the relevant date range. This representation makes evaluation robust to paraphrase while still preserving strong factual expectations.

3.4.4 Realism Constraints. The prompts that generate cases enforce several realism constraints. Queries never mention internal artifact identifiers directly. User identities are assigned plausibly, such as recipients for emails or attendees for meetings. Multi-hop cases are only produced when the source context contains a realizable dependency chain, and report cases are grounded in contiguous event windows so that the requested narrative exists in the tenant history. These constraints matter because they prevent the generator from creating benchmark cases that are formally valid but operationally unnatural.

3.4.5 Generation Pipeline. Case generation itself proceeds in three passes over the tenant history. The first pass shuffles artifact summaries and generates needle-in-a-haystack queries. The second pass groups events by day and proposes multi-step cases only when there are enough linked artifacts to support a dependency chain. The third pass samples 3–7 day windows and generates report requests over the resulting event clusters. By writing all cases to a single evaluation dataset with shared structure, EABench ensures that retrieval, reasoning, and report synthesis can be evaluated side by side under the same runtime and judge configuration.

3.5 Multi-Dimensional Evaluation Harness

3.5.1 Judge Configuration via YAML. The evaluation harness is configured independently from the runtime under test. A separate YAML file specifies the judge model, decoding parameters, and prompt templates for each scoring function. This separation is intentional. It allows a stronger or more stable model to serve as the judge, and it lets researchers redefine the evaluation criteria without editing the evaluation code. In

```

1 name: "Default Judge Prompts"
2 model:
3   provider: azure
4   name: gpt-4o
5   parameters: {temperature: 0.0}
6 prompts:
7   assertion_check: |
8     Verify whether the response satisfies each assertion.
9   citation_relevance: |
10    Judge the relevance of tool results to the query.
11   response_citation: |
12    Judge whether cited IDs are real, relevant, and
13    grounded in the tool trace.

```

Fig. 8. Condensed judge configuration illustrating independent control of the evaluation model and prompt templates.

practice, we recommend using a deterministic judge configuration with temperature fixed at zero so that repeated runs remain comparable.

3.5.2 Configurable Evaluation Metrics. EABench combines judge-based metrics with deterministic diagnostics. The assertion score measures factual coverage by checking whether the final answer satisfies the case assertions. Tool-search-result relevance measures whether the tools selected by the agent and the evidence they returned were appropriate for the query. Response-citation score measures whether the final answer cites real, relevant artifacts and uses a well-formed references section. These judge-based metrics are complemented by deterministic counts for retrieved evidence items, citations, latency, token usage, and tool calls. Because the prompt templates and thresholds are defined in YAML, researchers can extend the metric set or tighten the evaluation criteria without changing the harness logic.

Table 8. Metrics reported by the EABench evaluation harness.

Metric	Type	Interpretation
Assertion score	Judge-based	Fraction of natural-language assertions satisfied by the final answer
Tool search result relevance	Judge-based	Whether the selected tools and retrieved evidence were appropriate and faithfully reflected in the answer
Response citation score	Judge-based	Whether the final answer cites real, relevant artifacts and includes grounded references
Combined citation score	Derived	Mean of tool relevance and response citation scores
Tool search result number	Deterministic	Number of retrieved evidence items surfaced across search tool calls
Response citation number	Deterministic	Number of references emitted by the agent in the final answer
Latency, token usage, tool-call count	Deterministic	Operational cost and efficiency of the agent trajectory

3.5.3 Pass/Fail Criteria. A case is marked as passed only when two conditions hold simultaneously: the assertion score exceeds a correctness threshold and the response-citation score exceeds a grounding threshold. This dual criterion encodes an important design principle. A response that sounds plausible but cites

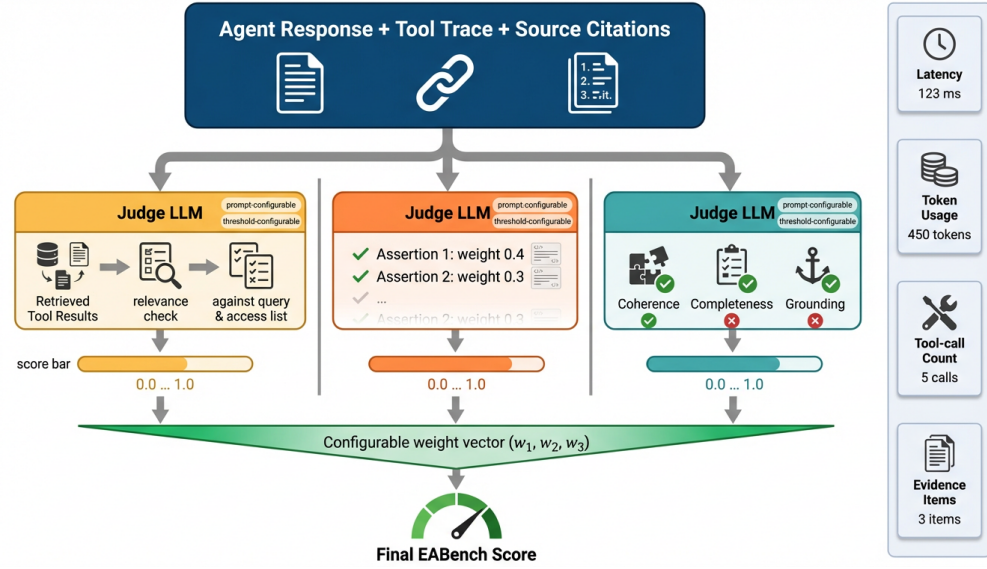


Fig. 9. EABench multi-dimensional evaluation harness. Three independently configurable LLM-judge tracks score citation relevance, assertion satisfaction, and response clarity; their outputs merge via a configurable weight vector into a single final score. Deterministic diagnostics complement the judge scores for operational profiling.

hallucinated artifacts should fail, and a fully cited response that does not answer the question should also fail. EABench therefore treats correctness and grounding as jointly necessary conditions for enterprise-agent success.

3.5.4 Side-by-Side Comparison and Diagnostics. For comparative studies, EABench can evaluate two agent configurations on the same case and ask a judge model to declare a winner. These pairwise judgments support win-rate analyses and Bradley–Terry modeling, while the underlying per-case metric vectors support paired tests such as the Wilcoxon signed-rank test. Because the result structure stores the full response, tool trace, assertion outcomes, and citation logs, comparative findings are accompanied by diagnostics that explain why one agent won rather than only whether it won.

3.5.5 User-Level Experience Simulation. Evaluation is always grounded in a requesting user context. Before each case is run, the search engine is bound to `case.user_id`, and access-control rules are enforced consistently across search results, citation verification, and debugging views. This design allows the same tenant to support engineer, manager, and contractor personas without maintaining separate corpora, and it turns access control into a benchmarked capability rather than an external precondition.

3.6 Interactive Debugging and Chat Interface

3.6.1 Live Chat Interface. Batch evaluation is necessary for aggregate comparison, but it is insufficient for rapid iteration on agent design. EABench therefore ships an interactive Streamlit application (Figure 10) in which the same runtime used for evaluation can be exercised through a live chat interface. A researcher selects an agent configuration, a tenant dataset, and a user persona from the sidebar, then issues free-form

Manuscript submitted to ACM

Table 9. Debugging surfaces exposed by the interactive workflow.

Surface	What it reveals
Agent & tenant selectors	Switch agent configuration and corpus without restarting
User persona login	Role, user ID, and access scope active for the current session
Inline citations	Numbered markers in the response linking to specific source artifacts
References panel	Full metadata (type, author, date, entity ID) for each cited artifact
Side-by-side comparison	Contrast two agent configurations on the same query and persona
Batch evaluation view	Aggregate metrics and per-case drill-down for completed eval runs

queries to inspect the agent’s behavior under that persona’s access scope. The main panel displays the agent’s structured response with inline citation markers and a references section listing source artifacts with metadata (type, author, date, entity ID). This mode is especially valuable during prompt engineering and tool-ablation studies, because it shortens the loop between a configuration change and a concrete behavioral observation.

3.6.2 Debugging Facilities. The interactive layer exposes the pieces of evidence that matter for debugging enterprise agents. The sidebar navigation provides four views: *Chat* for live query exploration, *Evaluation* for batch result inspection, *Side-by-Side Comparison* for contrasting agent configurations on the same query, and *Data Generator* for creating new tenant datasets. In the Chat view, the agent’s response includes inline citation markers that link to a References section with full artifact metadata—type, author, date, and entity ID—enabling a researcher to verify each cited source against the tenant corpus. The Login panel displays the active user’s role and ID, making access-scope constraints transparent. These views make it possible to distinguish several common failure modes: the agent may choose the wrong tool, retrieve weak evidence, lose track of a multi-hop chain, or cite the right artifact incorrectly. Because these signals are surfaced directly, EABench supports debugging by diagnosis rather than by prompt trial-and-error alone.

3.6.3 User Experience Testing. The interactive workflow is also useful for evaluating enterprise usability beyond aggregate benchmark scores. Researchers can replay the same case under multiple personas, inspect which artifacts become visible or hidden, and compare how different runtime configurations explain uncertainty to the user. In this sense, the interactive interface serves as a bridge between formal benchmarking and deployment-oriented validation: it preserves reproducibility while surfacing the qualitative details that determine whether an enterprise agent is trustworthy in practice.

4 Benchmark Scenarios and Instantiations

To demonstrate how EABench instantiates distinct enterprise workloads without changing the underlying codebase, we define three scenario families that vary organizational structure, confidentiality boundaries, and temporal reasoning demands. The purpose of these scenarios is not merely to populate different datasets; each scenario is designed to stress a different slice of agent capability, including access-aware retrieval, multi-hop reasoning across modalities, and long-form synthesis over multi-day windows. Table 10 summarizes the intended stressors.

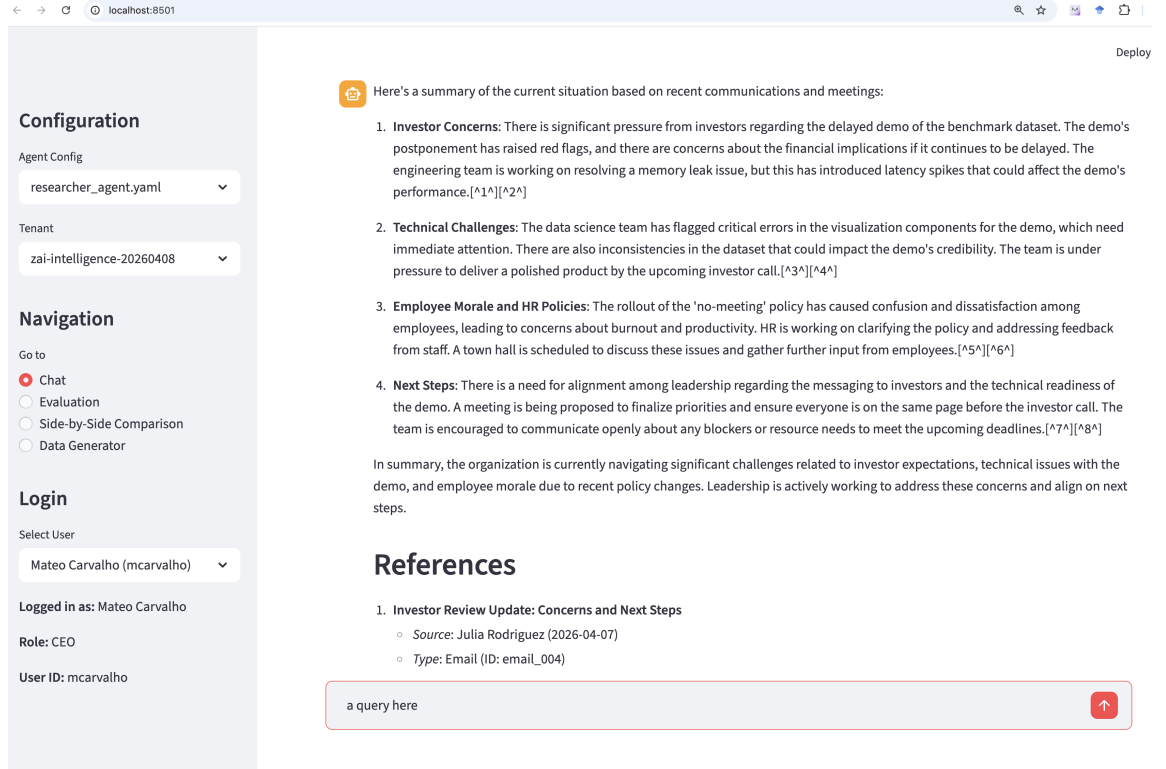


Fig. 10. EABench interactive chat interface. The left sidebar provides configuration controls (agent strategy, tenant, and user persona); the main panel displays the agent's structured response with inline citation markers and a references section listing source artifacts with metadata. A query input box at the bottom enables iterative exploration under the selected persona.

Table 10. Scenario families used to instantiate EABench and the benchmark capabilities they emphasize.

Scenario	Primary stressor	Dominant artifacts	Representative evaluation focus
Education	Policy evolution across administrative silos	Committee emails, meeting transcripts, planning documents, service-status updates	Timeline reconstruction for curriculum changes, attribution of administrative decisions, and deep-dive reports spanning outages and policy revisions
Law firm	Strict confidentiality and role-dependent visibility	Client briefs, internal legal memoranda, partner chats, case meetings	Access-control compliance, privilege-sensitive retrieval, and citation-grounded answers under consultant and attorney personas
Software company	Cross-modal incident response and product execution	Incident reports, engineering chats, sprint meetings, launch documents, customer escalation emails	Multi-hop tracing from symptom to root cause, tool-heavy search trajectories, and report generation over short operational windows

4.1 Education: University Administrators and Faculty

The education scenario models a university-like environment in which decisions emerge through committee processes rather than single-owner workflows. This structure produces queries that require the agent to connect policy drafts, meeting discussions, and follow-up emails over multi-week windows. It is therefore particularly useful for evaluating deep-dive report generation and temporally grounded retrieval, because the correct answer often depends on reconstructing how a proposal evolved across multiple administrative groups rather than locating a single decisive artifact.

4.2 Law Firm: Consultants and Attorneys

The law-firm scenario emphasizes confidentiality and sharply differentiated access rights. Here the central question is not only whether the agent can retrieve relevant material, but whether it can do so while respecting the requesting user’s role. This scenario is especially effective for evaluating user-level experience simulation: the same natural-language request may be answerable for a partner, partially answerable for a staff attorney, and intentionally blocked for a consultant. As a result, the scenario exercises both the retrieval stack and the policy layer that sits on top of it.

4.3 Software Company: SDEs and Managers

The software-company scenario concentrates dense cross-references into short operational bursts such as outages, launch reviews, and rollback decisions. It is the most tool-intensive scenario because answers frequently require the agent to combine people search, chat retrieval, meeting lookup, and file inspection inside the same trajectory. This makes it a strong stress test for the difference between ReAct and plan-and-execute agents: the former can react quickly to local evidence, while the latter benefits when a multi-step dependency chain must be planned before execution.

4.4 Dataset Statistics

We instantiated EABench across three concrete tenant organizations spanning distinct enterprise domains: a boutique law firm (*Bertrand & Co.*, 41 users), a mid-size research university (*University of Cambford*, 62 users), and a fast-growing AI startup (*ZAI Intelligence*, 165 users). Each tenant was generated independently by the same pipeline—GPT-4o (Azure OpenAI, temperature 0.7)—but parameterized with a domain-specific company description, a set of key narrative events, and an organizational size. The generation process follows a multi-stage design: (i) a daily story prompt produces a narrative arc from recent and long-range organizational history, (ii) summary prompts derive per-artifact synopses for each communication modality, and (iii) content prompts elaborate those summaries into full email bodies, chat transcripts, meeting minutes, and file contents. This staged approach ensures that artifacts within a single simulated day are mutually coherent—an email thread’s content is consistent with the meeting that prompted it—while still allowing realistic variation across days.

Table 11 summarizes the resulting artifact counts and evaluation-case distribution. For each tenant, three evaluation query generators—search, multi-hop, and report—produce structured test cases in a separate pass that examines the completed corpus and constructs questions, gold assertions, and entity lists against it.

Table 11. Artifact composition and evaluation case counts for the three EABench tenants. For chats and group chats, the count reflects distinct conversation threads rather than individual messages. Evaluation cases span search, multi-hop, and report query types (see Section 5.1.2).

Tenant	Domain	Users	Emails	DMs	Group	Meetings	Files	Eval Cases		
								Search	Multi-hop	Report
Bertrand & Co.	Legal	41	132	56	118	69	70	80	43	39
Univ. of Cambford	Education	62	120	41	118	70	70	80	44	38
ZAI Intelligence	AI / SW	165	298	140	301	164	164	79	82	40
Total		268	550	237	537	303	304	239	169	117

The three tenants vary substantially in scale: ZAI Intelligence is roughly four times larger than Bertrand & Co. across every artifact category, and its eval-case count (201) is proportionally expanded to maintain per-tenant statistical power. This heterogeneity is intentional—EABench is designed to stress-test agents under differing information densities rather than to standardize them away.

The query-type distribution follows the pipeline’s target split of approximately 40/40/20 (search/multi-hop/report). The actual ratios deviate slightly because each generator produces variable-sized batches. Search queries each reference exactly one gold entity, multi-hop queries reference 2–5 entities from different modalities, and report queries reference 3–15 entities spanning multiple artifact types and dates.

Table 12. Ground-truth evaluation complexity by query type and tenant. “Assertions” are gold-standard factual statements the agent’s response must satisfy; “Entities” are the ground-truth artifacts (emails, chats, meetings, files) required to answer the query.

Tenant	Query Type	Assertions per Case			Entities per Case		
		Mean	Med.	Range	Mean	Med.	Range
Bertrand	Fact-Check	2.4	2	2–4	1.0	1	1–1
	Multi-Hop	2.3	2	2–3	2.7	2	2–5
	Report	3.2	3	3–4	5.3	5	4–10
Cambford	Fact-Check	2.6	3	2–3	1.0	1	0–1
	Multi-Hop	2.8	3	2–4	3.1	3	2–4
	Report	3.2	3	3–4	5.9	5	3–15
ZAI	Fact-Check	2.2	2	2–3	1.0	1	1–1
	Multi-Hop	2.3	2	2–5	2.4	2	2–5
	Report	3.2	3	3–4	6.0	5	3–11

Table 12 reveals a clear complexity gradient across query types. Fact-check queries average 2.2–2.6 assertions grounded in a single entity, making them the simplest evaluation targets. Multi-hop queries maintain a similar assertion count but require cross-referencing 2–5 entities from different communication modalities. Report queries demand the most complex synthesis: 3–4 assertions each backed by 3–15 source entities spanning emails, chats, meetings, and files. This structured complexity scaling ensures that the evaluation set probes agent capabilities at multiple difficulty levels.

4.5 Communication Network Properties

To characterize inter-personal communication structure, we extract a directed multigraph for each tenant in which nodes are users and edges represent recorded communicative acts across five channel types: email (*from*→*to*), email CC (*from*→*cc*), direct-message chat (*sender*→*recipient*), group chat (*sender*→all participants), and meeting (pairwise edges among all attendees plus *organizer*→*attendee* edges). Table 13 reports standard graph-theoretic properties for each tenant’s aggregate communication network.

Table 13. Communication network topology for each tenant. Density = $|E|/(|V|(|V|-1))$ for a directed graph. Reciprocity is the fraction of directed edges that have a reverse edge. Transitivity is the global clustering coefficient. γ is the exponent of the power-law fit $P(k) \sim k^{-\gamma}$ to the in-degree complementary cumulative distribution (CCDF), with R^2 the goodness-of-fit.

Tenant	$ V $	$ E $	Density	Diam.	Avg. Path	Recip.	Trans.	γ	R^2
Bertrand	41	322	0.196	3	1.71	0.286	0.486	0.36	0.29
Cambford	62	519	0.137	4	1.89	0.343	0.448	0.68	0.42
ZAI	165	1,956	0.072	4	2.10	0.314	0.367	0.66	0.59

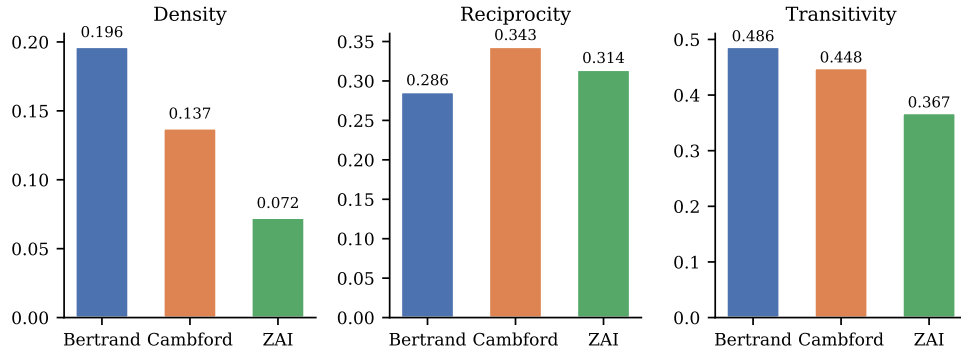


Fig. 11. Key communication-network metrics across the three EABench tenants. Density, reciprocity, and transitivity decrease with organizational scale, reflecting the transition from tight-knit professional-services firms to large engineering organizations with departmental silos.

These metrics reflect realistic organizational archetypes. Bertrand & Co., the smallest organization, is the densest (0.196) and has the smallest diameter (3), consistent with a tight-knit professional-services firm where most colleagues interact directly. ZAI Intelligence, the largest, exhibits a sparse, long-tail structure (density 0.072, 1,956 edges among 165 nodes) typical of rapidly growing technology companies with departmental silos. The University of Cambford occupies an intermediate position with the highest reciprocity (0.343), reflecting the collaborative back-and-forth characteristic of academic committee work. Transitivity is highest for Bertrand (0.486), where shared client matters create strong triadic closure, and lowest for ZAI (0.367), where cross-functional interaction is sparser.

The in-degree distributions follow a heavy-tailed pattern. We fit a power law $P(k) \sim k^{-\gamma}$ to the complementary cumulative distribution of in-degrees and compare the empirical graph against a synthetic Barabási–Albert (BA) preferential-attachment graph of the same size. The low γ values (0.36–0.68) confirm that all three networks exhibit right-skewed degree distributions rather than Poisson-like regularity. The R^2

values increase with tenant size (0.29 for Bertrand, 0.59 for ZAI), indicating that the power-law fit improves as the number of nodes permits a wider degree range. In each case, the maximum in-degree substantially exceeds the BA model’s prediction (e.g., 99 vs. 72 for ZAI, 43 vs. 34 for Cambford), indicating that the generated organizations produce realistic “hub” individuals—such as executives, team leads, or central administrative staff—whose communication connectivity is disproportionately high.

Per-channel decomposition reveals further structural realism. Email communication has reciprocity between 0.48 and 0.73 across tenants, reflecting the natural tendency for emails to receive replies. Chat messages are entirely non-reciprocal at the edge level (0.0), since each message is a directed act from sender to recipient with no guaranteed return in the same thread. Meeting edges, constructed from attendee co-presence, are fully reciprocal by construction (1.0). This channel-level heterogeneity is important for evaluation: an agent that reasons about “who communicated with whom” must integrate evidence from channels with qualitatively different relational semantics.

4.6 Temporal Communication Patterns

Realistic enterprise datasets exhibit bursty temporal structure: communication tends to cluster around deadlines, incidents, or scheduled events rather than arriving as a uniform Poisson process. We quantify this using the burstiness parameter $B = (\sigma_\tau - \mu_\tau) / (\sigma_\tau + \mu_\tau)$, where μ_τ and σ_τ are the mean and standard deviation of the inter-event time (IET) sequence for a given communication channel [?]. Values of B near zero indicate exponential (Poisson) inter-arrival times, while $B > 0$ indicates bursty clustering. We additionally report the memory coefficient M [?], which measures the correlation between consecutive IETs: $M > 0$ indicates that long gaps tend to follow long gaps (temporal clustering at multiple scales), while $M \approx 0$ indicates uncorrelated consecutive intervals.

Table 14. Inter-event time (IET) statistics per communication channel and tenant. Burstiness $B > 0$ indicates clustered arrivals; $B \approx 0$ indicates near-Poisson behavior. Memory coefficient M measures serial correlation between consecutive IETs. α is the power-law exponent fitted to the IET distribution. Meeting channels are omitted for brevity (their $|B| < 0.01$ in all tenants except ZAI, where $B = 0.23$ with $M = 0.19$).

Tenant	Channel	Events	B	M	α	Med. IET (s)
Bertrand	Email	132	−0.051	−0.054	2.00	81,450
	Chat (1:1)	1,022	+0.617	−0.055	0.33	72
	Group Chat	2,291	+0.560	−0.078	0.36	120
Cambford	Email	120	+0.065	−0.045	1.49	85,500
	Chat (1:1)	732	+0.640	−0.048	0.35	90
	Group Chat	2,075	+0.575	−0.072	0.34	110
ZAI	Email	298	+0.168	+0.395	1.28	82,800
	Chat (1:1)	2,581	+0.658	−0.042	0.33	77
	Group Chat	6,033	+0.614	−0.055	0.31	150

The results validate that EABench’s generation pipeline reproduces two hallmarks of real organizational communication. First, email arrivals are near-Poisson ($B \approx 0$) across all three tenants, with high α values (1.28–2.00) indicating that inter-arrival times decay rapidly—consistent with the scheduled nature of formal correspondence. Second, chat and group-chat streams are highly bursty ($B \in [0.56, 0.66]$) with low power-law exponents ($\alpha \in [0.31, 0.36]$), confirming heavy-tailed inter-event time distributions where short bursts

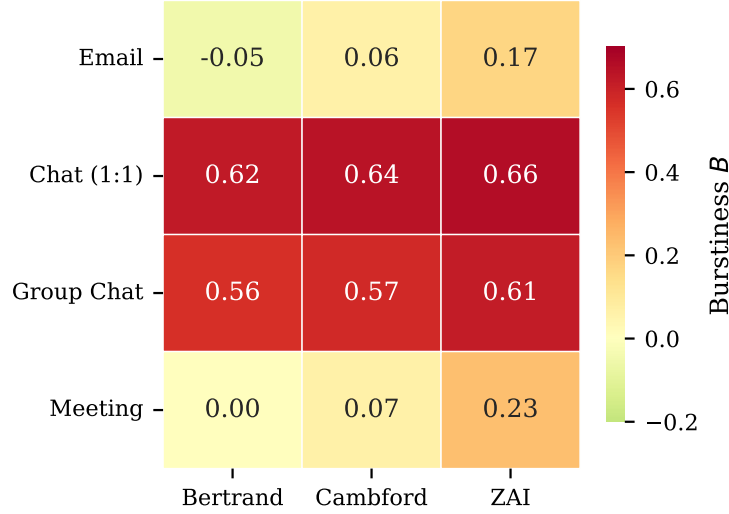


Fig. 12. Temporal burstiness B per communication channel and tenant. Chat and group-chat channels exhibit strong burstiness ($B > 0.5$) across all tenants, while email and meeting channels are near-Poisson ($B \approx 0$). ZAI’s email channel shows mild burstiness ($B = 0.17$) driven by correlated incident-response threads.

of rapid-fire messages are separated by long silences. This pattern matches empirical measurements of instant-messaging platforms in organizational settings [?].

The memory coefficients are near zero for most channel–tenant pairs ($M \in [-0.08, 0.0]$), indicating that the IET process is approximately memoryless at the single-channel level—as expected when daily narrative generation does not explicitly model autoregressive temporal dependence within a channel. The notable exception is ZAI’s email channel ($M = +0.40$), where the organizational scale produces enough correlated event clusters (e.g., multi-day incident threads, product-launch email bursts) to induce measurable serial dependence.

This temporal structure directly impacts retrieval difficulty: an agent must reason about *when* information was generated, not only *what* it contains, because a correct multi-hop answer often depends on reconstructing an event sequence from artifacts whose timestamps are concentrated in bursty clusters rather than distributed uniformly.

5 Experiments

EABench is designed to support controlled experimentation rather than a single fixed leaderboard. We organize this section around the experimental protocol: a description of the setup and metric definitions, followed by empirical results across three agent configurations and three tenant scenarios, ablation studies isolating the contribution of architectural and prompt-engineering choices, and an analysis of the cost-effectiveness frontier. Table 15 summarizes the core experimental design.

Table 15. Core experimental protocol supported by EABench.

Axis	Controlled variables	Primary outputs	Purpose
Cross-model evaluation	Tenant generator, runtime model, judge model, flow strategy	Pass rate, assertion score, citation score, win rate	Detect self-preference bias; verify ranking stability across generator–judge combinations
Ablation studies	Architecture (ReAct vs. plan-and-execute), prompt verbosity, execution temperature, tool configuration	Metric deltas per agent–tenant cell	Attribute performance to specific platform components
Scenario analysis	Industry template, company size, event density, persona mix	Per-scenario score distribution, failure taxonomy, latency diagnostics	Determine which scenario characteristics most affect retrieval, reasoning, and citation

5.1 Experimental Setup

We instantiate the three scenario families from Section 4 as concrete tenants (Bertrand & Co., University of Cambford, and ZAI Intelligence) and evaluate three agent configurations that span a spectrum from minimal reactive reasoning to deliberative planning. All agents share the same embedding model (`text-embedding-ada-002`) and use GPT-4o-family models via Azure OpenAI: the two ReAct agents use GPT-4o while the Researcher agent uses GPT-4o-mini for both planning and execution. The automated judge is GPT-4o at temperature 0.0, evaluating each case via four separate LLM calls that measure assertion satisfaction, tool-search relevance, and response-citation quality. Each agent is evaluated on all 525 cases (162 per small tenant, 201 for ZAI) covering three query types: search, multi-hop, and report. We record both judge-based quality metrics and deterministic operational diagnostics—latency, token usage, tool-call count, and retrieval volume—to distinguish capability failures from operational inefficiency.

5.1.1 Agent Configurations. We evaluate three agent configurations whose designs span a spectrum from minimal reactive reasoning to deliberative multi-step planning. All three share the same embedding model (`text-embedding-ada-002`) and use GPT-4o-family models via Azure OpenAI: the two ReAct agents use GPT-4o while the Researcher agent uses GPT-4o-mini. This design varies architecture, prompt engineering, and model size simultaneously, reflecting the realistic design space in which practitioners choose agent configurations.

Table 16. Configuration summary for the three evaluated agents. “Planning model” refers to the LLM used for the optional pre-execution planning step. Prompt size is the approximate system-prompt token count. Avg. tool calls and latency are empirical means across all evaluation cases.

Agent	Architecture	Model	T	Planning Model	Prompt Size	Avg. Tools	Avg. Lat. (s)
REACT-v1	Single-pass	GPT-4o	0.7	—	~3k tok	1.70	17.6
REACT-v2	ReAct						
	Single-pass	GPT-4o	0.5	—	~1.5k tok	1.70	18.2
RESEARCHER	Plan + Execute	GPT-4o-mini	0.0	GPT-4o-mini ($T=0$)	~3k tok	4.85	65.2

REACT-v1 implements a standard ReAct loop [21] with a verbose system prompt ($\sim 3,000$ tokens) containing eight fully worked THOUGHT $\rightarrow$ TOOL examples. These examples concretely demonstrate several critical behavioral patterns: dual-term person lookup (searching simultaneously by display name and by user ID), pronoun-to-ID resolution before issuing any content search, multi-query decomposition for relationship-based questions (“emails between A and B”), and detailed citation formatting with per-fact inline superscripts and a mandatory **## References** section listing artifact IDs. Its execution temperature of 0.7 permits exploratory variation in search-query phrasing.

REACT-v2 shares the same single-pass ReAct loop but condenses the system prompt to $\sim 1,500$ tokens across five terse bullet-style examples. The instructions cover the same topics—people resolution, citation formatting, pronoun disambiguation—but omit the extended worked demonstrations, replacing them with one-line directives. Its execution temperature is lowered to 0.5.

RESEARCHER introduces a dedicated planning stage: GPT-4o-mini (at temperature 0.0) first generates a structured, step-by-step research plan before any tool call is issued. The planning prompt explicitly requires four capabilities: (i) *multi-hop reasoning*—decomposing complex questions into dependency chains; (ii) *correlated search*—cross-referencing across emails, chats, meetings, and files; (iii) *mandatory ID resolution*—resolving names to user IDs via `search_people` before any content search; and (iv) *step-by-step ordering*—arranging steps so that the output of each feeds the next. The planner is instructed to output only the plan, not to execute it. The same GPT-4o-mini model (at temperature 0.0) then executes the plan step by step using the full tool set, resulting in systematically broader retrieval (61.6 items retrieved per case vs. 15–19 for the reactive agents) and substantially higher token consumption ($\sim 92k$ prompt tokens vs. $\sim 18k$). Notably, despite using a smaller model than the ReAct agents, the Researcher achieves the highest pass rate, demonstrating that structured planning can compensate for reduced per-call model capacity.

5.1.2 Evaluation Metrics. Each evaluation case consists of a natural-language query, a requesting user identity, a list of gold-standard factual assertions, and a list of entity references that a correct answer should cite. Three query generators produce cases of increasing complexity:

- (1) **Search queries** target the retrieval of specific items with filtering constraints (e.g., “Find emails sent by Lisa about fund Beta in the last two weeks”).
- (2) **Multi-hop queries** require chaining evidence across artifact types and resolving entity relationships (e.g., “Who attended the meeting where the Q3 budget was discussed, and what follow-up action did they take by email?”).
- (3) **Report queries** request synthesis or summary generation over multi-day windows (e.g., “Provide a timeline of all decisions related to the server migration from March 1 to March 15”).

The automated judge (GPT-4o, temperature 0.0) computes four complementary metrics via separate LLM calls:

Assertion score. Each gold assertion a_i is verified independently against the agent’s response. The judge outputs a Boolean *True/False* per assertion together with a brief explanation, and the final assertion score for the case is the fraction of assertions satisfied: $\text{Assr.} = |\{a_i : \text{True}\}|/|A|$.

Tool-search relevance. The judge evaluates three aspects of the agent’s tool usage in a single call: (i) whether the tool calls were *appropriate* for the query, (ii) whether the retrieved evidence was *relevant* to the question,

and (iii) whether the final response is *grounded* in the retrieved artifacts rather than hallucinated. The result is a scalar relevance score in $[0, 1]$. We additionally record the number of items retrieved per case (*tool-search count*).

Response-citation quality. A separate judge call inspects the response’s **## References** section, checking (i) whether one exists, (ii) how many cited artifact IDs are *grounded* (i.e., appear in the tool-call results) versus *hallucinated* (fabricated IDs), and (iii) whether inline citation markers appear in the response body. The score is in $[0, 1]$; we also record the raw grounded-citation count.

Pass rate. A case is marked *passed* if the assertion score is 1.0 (all assertions satisfied). Pass rate is the fraction of cases that pass.

For aggregate reporting, we define a composite *citation score* as the mean of tool-search relevance and response-citation quality. This decomposition is important because the two sub-scores measure different failure modes: an agent can retrieve excellent evidence but fail to format citations (high tool-search, low response-citation), or it can produce well-formatted references that point to irrelevant material (low tool-search, high response-citation).

5.1.3 Main Results. Table 17 reports the decomposed metrics across all nine agent–tenant combinations. Table 18 aggregates results across tenants.

Table 17. Per-tenant and per-agent evaluation results. *Pass*: pass rate. *Assr.*: assertion score. *TS-Rel*: tool-search relevance. *TS-n*: mean items retrieved. *RC-Rel*: response-citation relevance. *RC-n*: mean grounded citations. *Cite*: composite citation score (mean of TS-Rel and RC-Rel). *Lat.*: mean latency in seconds. Bold marks the best per-tenant value.

Tenant	Agent	Cases	Pass	Assr.	TS-Rel	TS-n	RC-Rel	Cite	Lat. (s)
Bertrand	REACT-v1	162	0.414	0.736	0.803	19.2	0.593	0.698	18.1
	REACT-v2	162	0.259	0.729	0.650	15.8	0.479	0.565	22.2
	RESEARCHER	162	0.432	0.625	0.762	61.1	0.640	0.701	71.0
Cambford	REACT-v1	162	0.228	0.515	0.717	18.3	0.449	0.583	17.3
	REACT-v2	162	0.222	0.489	0.701	17.0	0.440	0.571	15.8
	RESEARCHER	162	0.346	0.543	0.793	61.1	0.684	0.738	55.3
ZAI	REACT-v1	201	0.269	0.560	0.797	17.9	0.558	0.677	19.1
	REACT-v2	201	0.154	0.473	0.717	13.5	0.493	0.605	16.5
	RESEARCHER	201	0.289	0.503	0.765	62.5	0.623	0.694	69.1

Table 18. Aggregate agent rankings across all three tenants. Columns follow the same conventions as Table 17. *Prompt tok.*: mean input tokens per inference call. *Tool calls*: mean tool invocations per case.

Agent	Pass	Assr.	TS-Rel	TS-n	RC-Rel	RC-n	Cite	Lat. (s)	Tools	Prompt tok.
RESEARCHER	0.355	0.557	0.773	61.6	0.649	2.81	0.711	65.2	4.85	92,280
REACT-v1	0.323	0.638	0.799	18.5	0.551	1.87	0.675	17.6	1.70	17,778
REACT-v2	0.212	0.564	0.689	15.4	0.471	1.84	0.580	18.2	1.70	17,203

RESEARCHER achieves the highest overall pass rate (35.5%) and citation quality (0.711) but the lowest assertion score (0.557). REACT-v1 ranks second on pass rate (32.3%) while leading on assertion accuracy

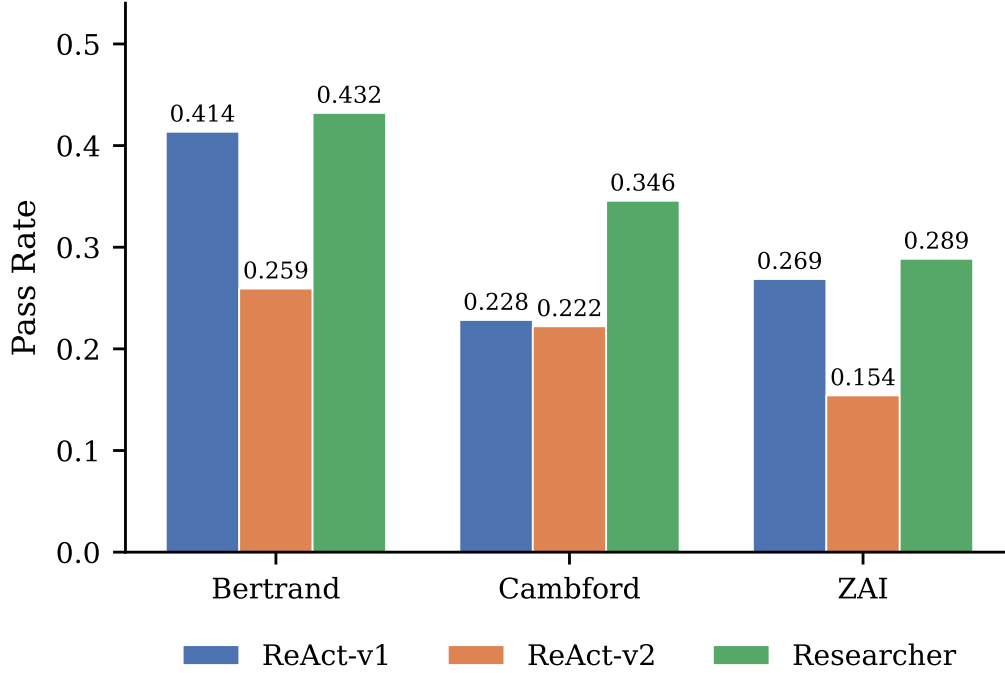


Fig. 14. Pass rate (fraction of cases with all assertions satisfied) by agent configuration and tenant. RESEARCHER leads on Bertrand and Cambford; the gap narrows on ZAI where evidence is temporally concentrated.

(0.638) and tool-search relevance (0.799). REACT-v2 trails on all quality metrics (pass 21.2%, citation 0.580) despite its structural similarity to v1. The decomposed citation scores reveal that RESEARCHER’s advantage is concentrated in response-citation quality (0.649 vs. 0.551 for v1), while REACT-v1 actually leads on tool-search relevance (0.799 vs. 0.773). This means that the reactive agent retrieves slightly more relevant evidence per tool call, but the planning agent produces better-structured reference sections because its broader retrieval footprint provides more citable material.

5.1.4 Effect of Agent Architecture: Plan-and-Execute vs. ReAct. The primary architectural difference between RESEARCHER and the two reactive agents is the separation of planning from execution. Before issuing any tool call, RESEARCHER prompts GPT-4o-mini to produce a structured research plan that decomposes the query into a dependency chain, identifies which data modalities to cross-reference, and mandates people-ID resolution before any content search. This design directly explains three empirical patterns.

First, RESEARCHER retrieves 61.6 items per case versus 15–19 for the reactive agents—a factor of $\sim 3.5\times$. The planning prompt explicitly requires correlated search across emails, chats, meetings, and files, whereas the reactive agents only pursue additional artifact types opportunistically. This broader retrieval footprint is the primary driver of the response-citation advantage: with more retrieved artifacts available, the execution model has more material to cite, raising the grounded-citation count from 1.87 (v1) to 2.81 (RESEARCHER).

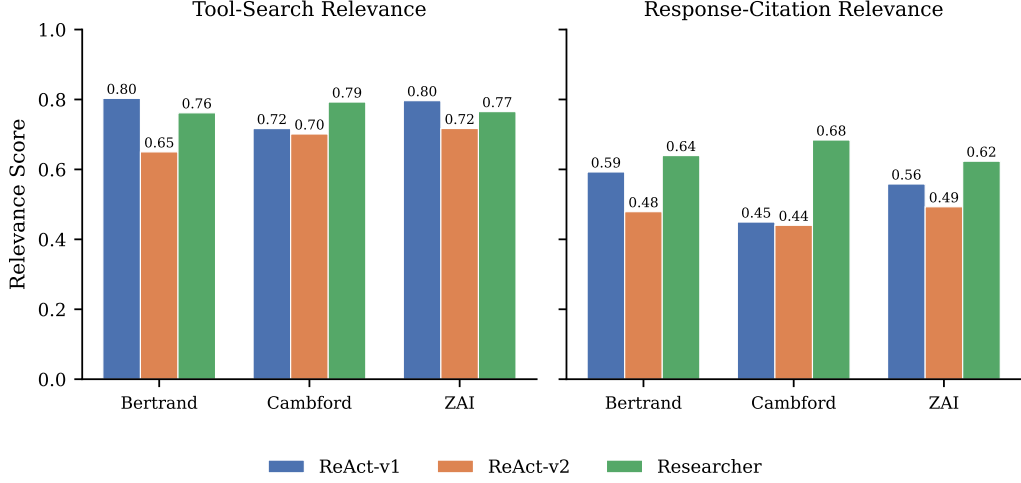


Fig. 15. Decomposed citation metrics. REACT-v1 leads on tool-search relevance (left) while RESEARCHER leads on response-citation quality (right), showing that retrieval precision and citation formatting are largely independent failure modes.

Second, the planning stage is most beneficial when queries require temporal multi-hop reasoning. On Cambford, where cases frequently require reconstructing policy evolution across weeks of committee emails and subsequent meeting threads, RESEARCHER improves pass rate by 52% relative to REACT-v1 (0.346 vs. 0.228) and achieves the highest citation score in any cell (0.738). On Bertrand, where the smaller, denser organization makes most answers reachable via a single well-formed search, the pass-rate gap shrinks to 4% (0.432 vs. 0.414). On ZAI, where evidence is concentrated in dense operational bursts, REACT-v1 closes to within 2 percentage points of RESEARCHER on pass rate (0.269 vs. 0.289) and substantially exceeds it on assertion score (0.560 vs. 0.503).

Third, despite leading on pass rate and citation, RESEARCHER lags on assertion score (0.557 vs. 0.638 for v1). Two factors contribute: the smaller GPT-4o-mini model has lower per-call reasoning capacity than GPT-4o, and the deterministic $T=0.0$ decoding constrains the model’s ability to flexibly integrate retrieved evidence into cohesive factual statements. The planning stage further compounds this by guiding execution toward broad retrieval at the expense of precise fact synthesis. The result is a coverage-vs.-precision trade-off: RESEARCHER achieves broad retrieval coverage at the cost of per-assertion accuracy.

5.1.5 Effect of Prompt Engineering: Verbosity and Temperature. REACT-v1 and REACT-v2 share the same architecture, backbone model (GPT-4o), and tool set, yet their aggregate pass rates differ by 11 percentage points (32.3% vs. 21.2%). Two configuration differences explain this gap.

The first is system-prompt verbosity. REACT-v1’s eight fully worked examples concretely demonstrate each behavioral pattern: the agent sees a complete THOUGHT→TOOL→OBSERVATION chain for dual-term person lookup, pronoun resolution, multi-query decomposition, Python analytics, and citation formatting. REACT-v2’s five bullet-style examples state the same requirements but as abstract directives rather than worked instances. The consequence is most visible in citation behavior: v2’s response-citation relevance

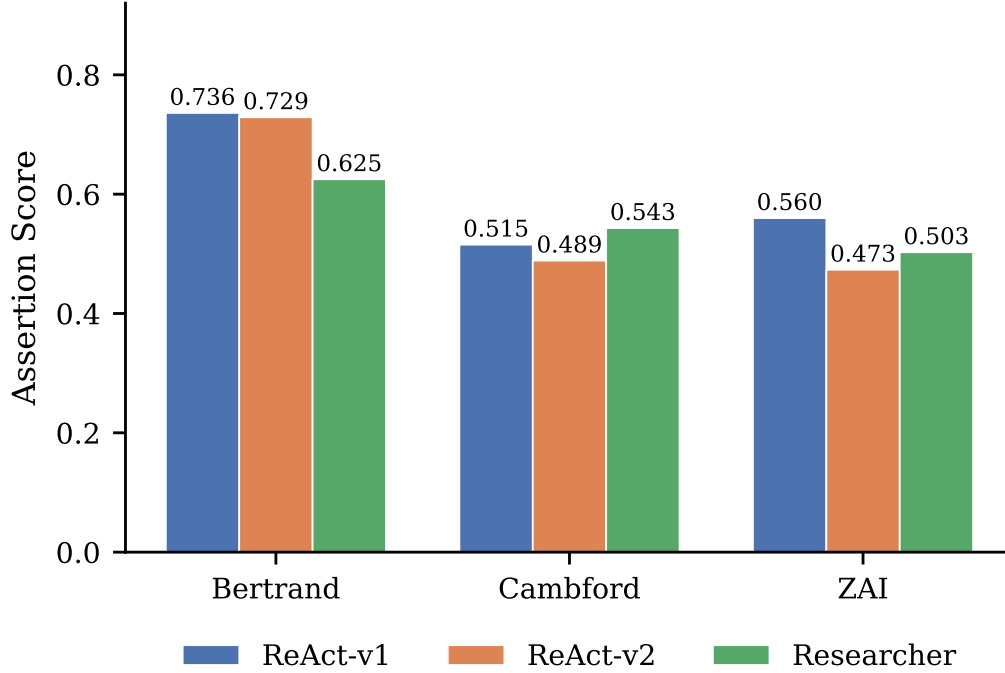


Fig. 16. Mean assertion score (fraction of gold assertions satisfied) by agent and tenant. REACT-v1 leads on factual precision across all three tenants, while RESEARCHER’s broad retrieval strategy sacrifices per-assertion accuracy for coverage.

(0.471) is 8 points below v1’s (0.551), suggesting that the model benefits more from seeing the correct citation format demonstrated than from being told to produce it. Tool-search relevance drops even more sharply (0.689 vs. 0.799), indicating that the condensed prompt also impairs the quality of the search queries themselves, not just the final formatting.

The second factor is execution temperature. At $T=0.7$, REACT-v1 introduces stochastic variation in search queries; slight rephrasings of the same semantic query can surface documents that a deterministic formulation misses due to embedding-space locality. At $T=0.5$, REACT-v2 converges more quickly to the most probable next action but misses edge cases requiring alternative search terms. The effect is amplified on ZAI, the most complex tenant, where v2’s pass rate collapses to 0.154—less than half of v1’s 0.269—suggesting that compressed prompt guidance combined with lower temperature is insufficient to handle high information density without planning support.

5.1.6 Scenario-Level Analysis. The three tenants stress different agent capabilities, and the per-tenant results expose the conditions under which each architecture excels or fails.

Bertrand & Co. (Legal). The law firm’s 41-user dense network (density 0.196, diameter 3) means that most entities are reachable within two hops. Even a single well-executed tool call frequently suffices to locate the relevant artifact. All three agents perform comparably on pass rate—REACT-v1 at 0.414 versus

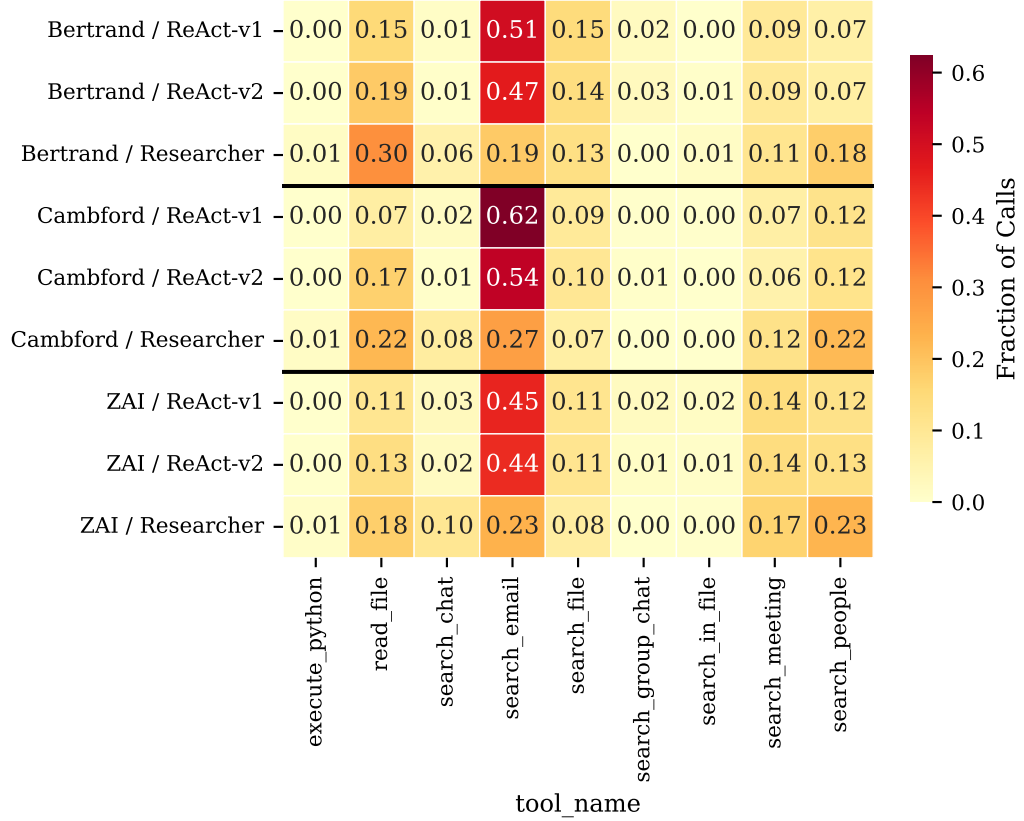


Fig. 17. Tool-call distribution by agent and tenant. Rows are grouped by tenant; columns show individual tools. RESEARCHER allocates calls across more tool types in every tenant, while the reactive agents concentrate on `search_messages` and `search_people`. Cross-tenant variation is visible: ZAI’s larger corpus drives higher `read_file` usage across all agents.

RESEARCHER at 0.432—and REACT-v1 leads decisively on assertion score (0.736 vs. 0.625). The planning overhead provides near-zero benefit when evidence is organizationally concentrated.

Univ. of Cambford (Education). This tenant is the most challenging for reactive agents. Policies evolve across multi-week committee processes; the agent must connect a policy draft in one email thread, a revision discussed in a later meeting, and a final decision communicated via group chat. Without an explicit dependency-chain plan, REACT-v1 achieves only 0.228 and REACT-v2 0.222—barely distinguishable—because both fail to systematically trace temporal chains. RESEARCHER’s planning stage raises pass rate to 0.346 and citation to 0.738, the highest across all nine cells. This scenario is the clearest demonstration of when plan-and-execute provides a qualitative advantage over reactive reasoning.

ZAI Intelligence (Software / AI). The AI-startup tenant combines the largest scale (165 users, 1,956 edges) with the highest chat burstiness ($B = 0.66$, Table 14). Evaluation cases often involve tracing an incident or product decision through a dense cluster of same-day engineering chats, follow-up emails, and

Manuscript submitted to ACM

sprint meetings. Here the reactive agents are competitive because the evidence is temporally concentrated and a well-targeted single search often retrieves several related artifacts simultaneously. REACT-v1 achieves 0.269 pass and 0.560 assertion score—within 2 points of RESEARCHER’s 0.289 pass rate but substantially above its 0.503 assertion score. This suggests that the planning overhead can be counterproductive when evidence is dense: the planner introduces additional, potentially irrelevant search steps that consume context budget without improving accuracy.

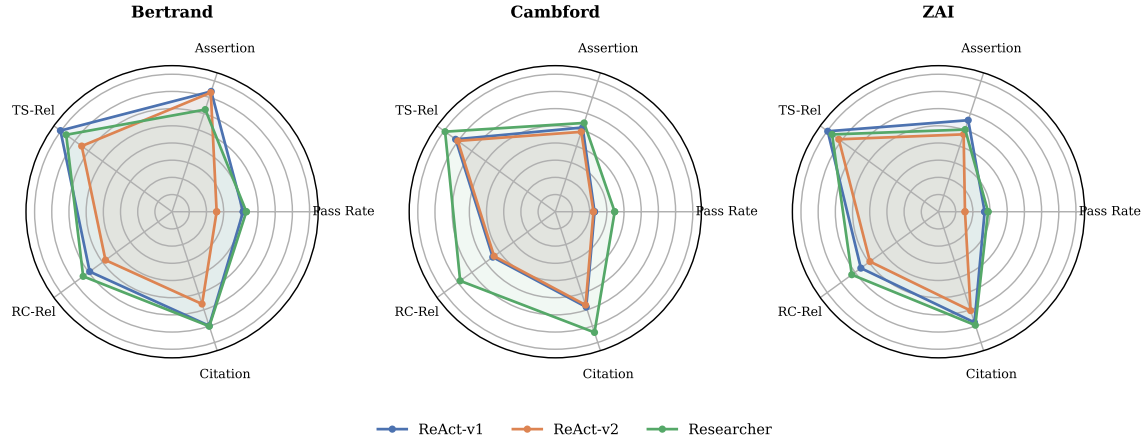


Fig. 18. Per-tenant agent profiles. Each subplot shows the three agents on a shared radar for one tenant. On Bertrand (dense, small) all agents overlap; on Cambford (temporal multi-hop) RESEARCHER’s citation and pass-rate advantage widens; on ZAI (large, bursty) REACT-v1 matches or exceeds the planner on assertion accuracy.

5.1.7 Query-Type Analysis. The evaluation set comprises three query categories generated with a 40/40/20 target split: *fact-check* (single-entity retrieval), *multi-hop* (cross-artifact reasoning), and *report* (multi-source synthesis). Breaking down assertion and citation metrics by query type reveals how each architecture handles increasing retrieval complexity.

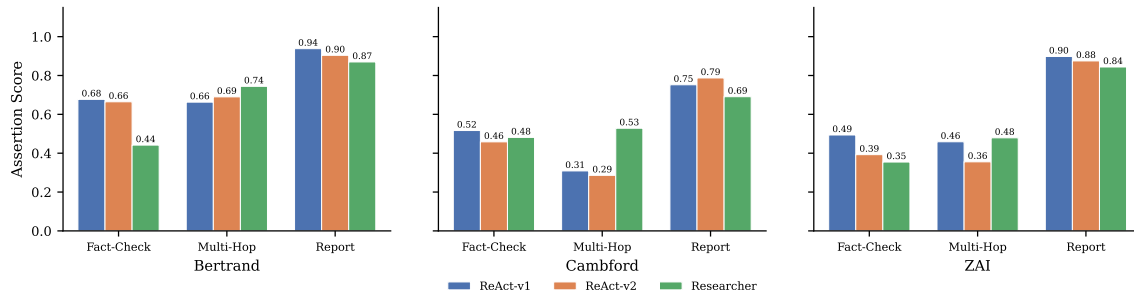


Fig. 19. Mean assertion score by query type and tenant. All agents perform best on fact-check queries. The gap between agents widens on multi-hop and report queries, where REACT-v1’s precision advantage is most pronounced.

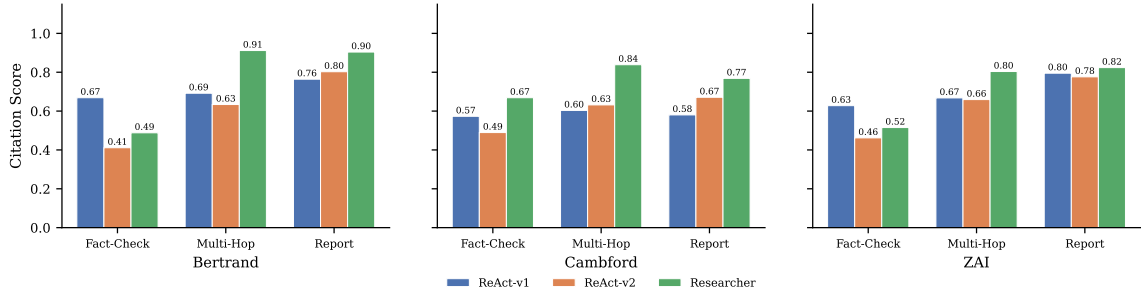


Fig. 20. Mean citation score by query type and tenant. RESEARCHER leads on report queries where its planning stage systematically covers multiple source types, while the reactive agents are competitive on fact-check queries.

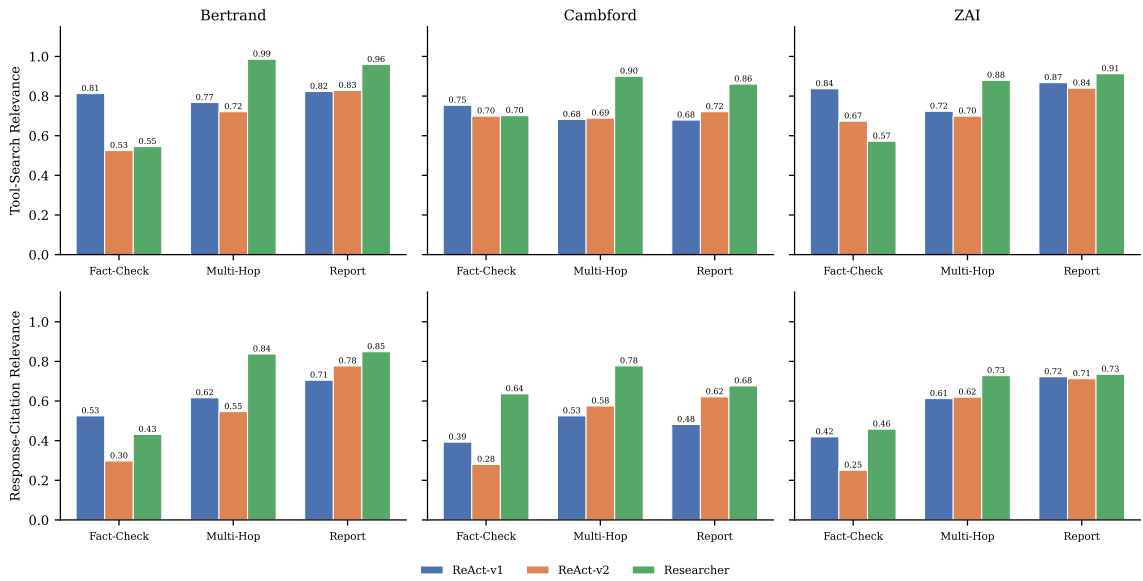


Fig. 21. Citation decomposition (tool-search relevance and response-citation relevance) by query type. Top row: tool-search relevance; bottom row: response-citation relevance. The performance gap between agents is most visible on report queries, where RESEARCHER's broader retrieval strategy produces higher relevance across both sub-metrics.

On fact-check queries, which require locating a single artifact, all three agents achieve their highest assertion scores and the inter-agent gap is narrow. This confirms that when evidence is concentrated in a single entity, even a minimal reactive loop suffices. On multi-hop queries, assertion scores drop across the board as agents must connect information across multiple artifacts; here REACT-v1's detailed worked examples help it maintain higher per-assertion accuracy than RESEARCHER, whose broader retrieval introduces noise. On report queries, RESEARCHER's planning stage provides the clearest advantage in citation quality: its dependency-chain decomposition ensures that multiple source types are systematically retrieved and cited, while the reactive agents struggle to organize comprehensive responses from ad-hoc retrieval sequences.

5.1.8 Cost-Effectiveness Trade-off. The results reveal a clear cost-effectiveness frontier. RESEARCHER achieves the highest pass rate (35.5%) and citation quality (0.711) but consumes $5.2\times$ more prompt tokens (92,280 vs. 17,778) and $3.7\times$ more wall-clock time (65.2s vs. 17.6s) than REACT-v1. For deployments where retrieval completeness and citation traceability are the primary objectives—legal research, compliance auditing, or due-diligence workflows—this overhead may be justified. For latency-sensitive applications or scenarios with dense, recent evidence, REACT-v1 provides 32.3% pass rate at $3.7\times$ lower latency with higher factual precision (assertion 0.638 vs. 0.557).

REACT-v2 does not occupy a Pareto-optimal point on this frontier: it matches REACT-v1 on tool-call count (1.70) and is marginally faster, but lags on all quality metrics. This finding suggests that prompt compression is costly in the absence of compensating architectural changes—a concise prompt is not equivalent to an equally effective one.

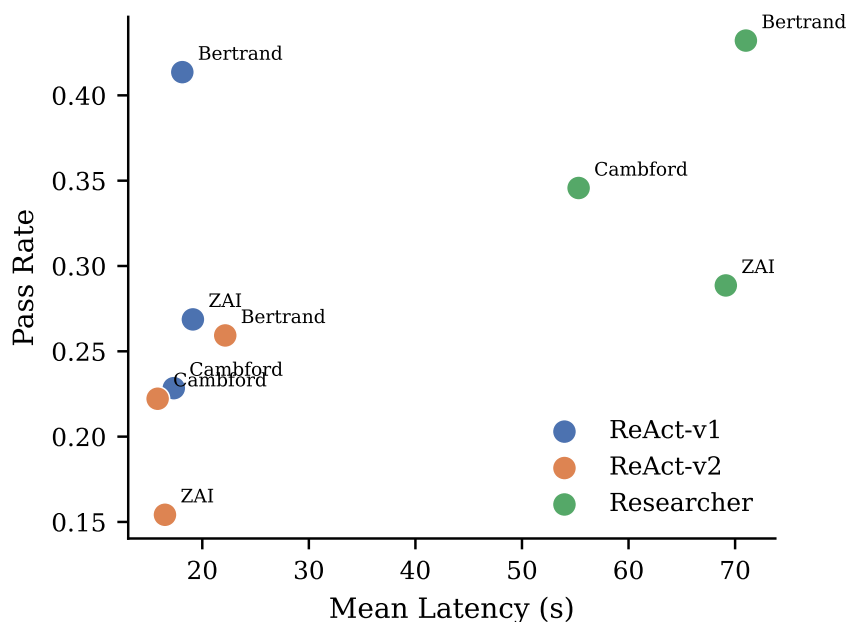


Fig. 22. Latency-quality trade-off: mean wall-clock time vs. pass rate. RESEARCHER occupies the high-latency/high-quality quadrant while the two reactive agents cluster at low latency; REACT-v1 achieves $3.7\times$ lower latency than RESEARCHER with only a modest pass-rate gap.

5.2 Cross-Model Evaluation and Bias Mitigation

A synthetic benchmark must guard against circularity: if the same model family generates the data, runs the agent, and judges the output, self-preference bias can inflate scores. The EABench protocol treats the generator, runtime, and judge as three independent factors. In the experiments reported above, the generator and judge both use GPT-4o while the runtime varies across three agent configurations that differ in architecture, prompt design, and model size (GPT-4o for the ReAct agents, GPT-4o-mini for the Researcher). This setup reflects the realistic practitioner design space in which architecture, prompting,

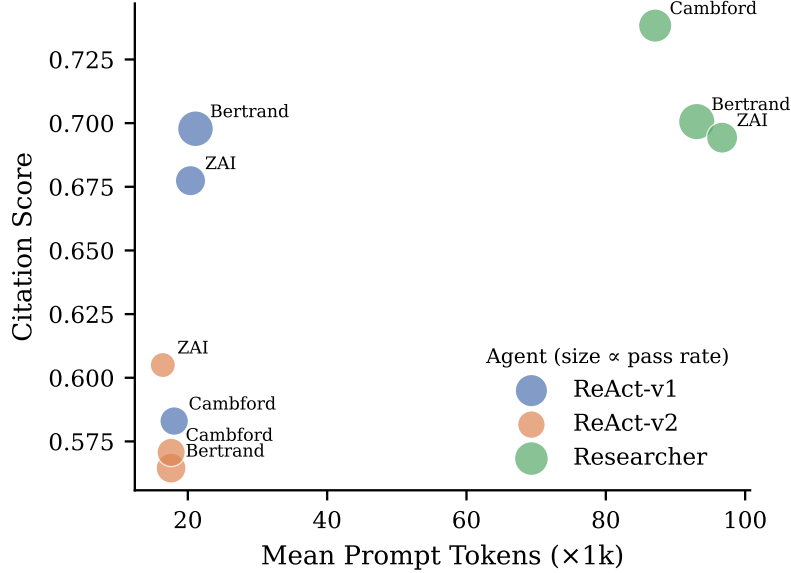


Fig. 23. Cost–quality frontier: mean prompt tokens vs. citation score. Marker size is proportional to pass rate. RESEARCHER achieves the highest citation quality at $5.2\times$ the token cost of the reactive agents; REACT-v2 does not occupy a Pareto-optimal point.

and model choice are varied jointly. To fully control for generator–judge bias, the framework supports a cross-model matrix in which tenants generated by model family G_i are evaluated by runtime R_j and judged by J_k , with the restriction that no conclusion is drawn from a single diagonal cell. Diagonal effects—where a model consistently scores higher only when generating or judging its own data—can then be identified as self-preference rather than genuine method improvement.

5.3 Discussion

The experiments expose three principal findings. First, the separation of planning from execution provides a measurable advantage on multi-hop temporal queries (52% relative pass-rate improvement on Cambford) but negligible benefit on organizationally concentrated evidence (4% improvement on Bertrand). This interaction between architecture and scenario structure is precisely the kind of diagnostic that a configurable benchmark is designed to surface. Second, prompt engineering matters as much as architecture: the 11-point pass-rate gap between REACT-v1 and REACT-v2—identical architectures differing only in prompt verbosity and temperature—exceeds the 3-point gap between REACT-v1 and RESEARCHER on two of three tenants. Third, the decomposed citation metrics reveal that retrieval quality and citation formatting are largely independent failure modes: REACT-v1 leads on tool-search relevance (0.799) while RESEARCHER leads on response-citation quality (0.649), indicating that broad retrieval and well-structured references require different capabilities.

These findings suggest that future work should explore hybrid architectures that conditionally activate planning based on query complexity, as well as prompt-optimization techniques that maintain the behavioral specificity of verbose prompts at lower token cost.

6 Conclusion and Discussion

EABench reframes enterprise-agent benchmarking as an end-to-end systems problem. Instead of providing only a fixed dataset or only a runtime abstraction, it joins four ingredients in a single platform: coherent tenant generation, configurable agent execution, configurable query and judge generation, and scenario-aware debugging under access control. This combination is what makes the benchmark useful for research rather than merely descriptive. It lets investigators ask not only whether one agent outperforms another, but why that difference appears and whether it survives changes in scenario, model family, retrieval stack, or judge configuration.

The framework still has clear limitations. It currently assumes English-language enterprise communication, depends on sufficiently capable foundation models to preserve long-range narrative coherence during generation, and relies on judge prompts whose scope must be reviewed carefully when researchers introduce new task families. In addition, interactive debugging is a complement to formal evaluation rather than a substitute for it: visual inspection can diagnose failure modes, but final comparisons must still be grounded in reproducible case-level metrics.

These limitations point directly to future work. The next priorities are broader multilingual support, richer sandbox backends for code-centric enterprise tasks, stronger controls against generator–judge self-preference bias, and expanded human validation protocols for both realism and scoring quality. Even in its current form, however, EABench provides a practical and extensible foundation for studying enterprise LLM agents under realistic data, realistic constraints, and experimentally meaningful variation.

References

- [1] Amirhossein Abaskohi, Tianyi Chen, Miguel Munoz-Marmol, Curtis Fox, Amrutha Varshini Ramesh, Etienne Marcotte, Xing Han Lu, et al. 2025. DRBench: A Realistic Benchmark for Enterprise Deep Research. *arXiv preprint arXiv:2510.00172* (2025).
- [2] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios N. Angelopoulos, Tianyi Li, Dacheng Li, et al. 2024. Chatbot Arena: An open platform for evaluating LLMs by human preference. *arXiv preprint arXiv:2403.04132* (2024).
- [3] Prafulla Kumar Choubey, Xiangyu Peng, Shilpa Bhagavath, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. 2025. Benchmarking Deep Search over Heterogeneous Enterprise Data. *arXiv preprint arXiv:2506.23139* (2025).
- [4] Open-Source Community. 2025. GAIA-2: The Next Generation of General AI Assistants Benchmark. *arXiv:2501.00000* [cs.AI]
- [5] A. Drouin, M. Gasse, M. Caccia, I. H. Laradji, M. Del Verme, T. Marty, et al. 2024. WorkArena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718* (2024).
- [6] E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, et al. 2024. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872* (2024).
- [7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *ICLR*.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [9] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, et al. 2023. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688* (2023).
- [10] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A Benchmark for General AI Assistants. In *ICLR*.

- [11] Microsoft. 2023. Microsoft 365 Copilot: The AI-powered productivity tool for work. Microsoft Blog.
- [12] A. Mitra, L. Del Corro, S. Mahajan, A. Coda, C. Simoes, S. Agrawal, et al. 2024. AgentInstruct: Toward generative teaching with agentic flows. *arXiv preprint arXiv:2407.03502* (2024).
- [13] Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. 2026. OfficeQA Pro: An Enterprise Benchmark for End-to-End Grounded Reasoning. *arXiv preprint arXiv:2603.08655* (2026).
- [14] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, et al. 2023. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789* (2023).
- [15] Divyanshu Saxena, Rishikesh Maurya, Xiaoxuan Ou, Gagan Somashekar, Shachee Mishra Gupta, Arun Iyer, Yu Kang, Chetan Bansal, Aditya Akella, and Saravan Rajmohan. 2025. Continuous Benchmark Generation for Evaluating Enterprise-Scale LLM Agents. *arXiv preprint arXiv:2511.10049* (2025).
- [16] Harsh Vishwakarma, Ankush Agarwal, Ojas Patil, Chaitanya Devaguptapu, and Mahesh Chandran. 2025. Can LLMs Help You at Work? A Sandbox for Evaluating LLM Agents in Enterprise Environments. In *EMNLP*.
- [17] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim. 2023. Plan-and-Solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *ACL*.
- [18] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, et al. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [19] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *EMNLP*.
- [20] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao, and K. Narasimhan. 2024. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045* (2024).
- [21] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao. 2022. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [22] W. Zhao, X. Ren, J. Hessel, C. Cardie, Y. Choi, and Y. Deng. 2024. WildChat: 1M ChatGPT interaction logs in the wild. In *ICLR*.
- [23] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, et al. 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS*.
- [24] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, S. Sridhar, et al. 2023. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).